

1009 Parent, Teacher, and Student Guide to Joystick-Uno-L298N

STEM Starter Kit Series, Companion to Articles 1000 to 1004 (Article 1009)



Invest in their hands today, and they will build our tomorrow.

Table of Contents

Purpose of This Guide
The Series at a Glance
Safety and Supervision
Programming Over Wiring
Materials
The Electronics Lab
Power Supply
Choosing the Motor Bus Voltage
Measuring the Motor Bus and the L298N Drop
The Author's Bench
Wiring
Article Summaries
Article 1000, Introduction Robotics (Part 1)
Article 1001, Joystick Algorithm (Part 2)
Article 1002, Arduino Uno: Pins, Ports, and Peripherals (Part 3)
Article 1003, L298N Motor Driver (Part 4)
Article 1004, Experiments for Joystick-Uno-L298N (Part 5)
The Signal Chain
Uno Pin Assignments
Setup Procedures
Why Setup Is Done in Two Stages
Joystick Setup
L298N Setup

Motor Movement Checklist
Troubleshooting
Appendix, Variable Bench Power Supply with Variac
Purpose of the Variac Supply
Circuit-2, the Schematic
Image-8a, the Bench Model
Using the Variac Supply
Advanced Questions with the Variable Supply
A Note on Software and Firmware
Bibliography
Disclaimer and License
Correspondence

The Two Setup Procedures

Joystick Setup and L298N Setup are the most important procedures in this guide. Together they match the program to the wiring, so that the student corrects the behavior of the project in software and never by moving wires.

Joystick Setup, with Code-1, corrects the joystick in software. The student reads the two joystick values on the serial monitor and, with no motors connected, corrects any exchanged or reversed axis by changing one line of the program.

L298N Setup, with Code-2 and Image-5 to Image-7, corrects the motors in software. The student places one value of the BitsL298N enumeration, from bits_0000 to bits_1111, in the Bits() call of Code-2, uploads it, and checks the motor movement with the Motor Movement Checklist. The student moves to the next value at the first wrong movement, and stops at the first value that passes all eight joystick positions.

Thumb joystick modules from different manufacturers do not have the same circuit. The student therefore treats the joystick as a black box, as in Article 1000: the push button is first tested with the Button class as supplied (Joystick Setup, step 4), and the circuit of the module is studied only if that test fails.

Each setup procedure is completed and recorded in the notebook before the next one begins.

List of Figures and Tables

Image-1	The author's earlier joystick board
Image-2a	The Electronics Lab
Image-2b	The Electronics Lab, labeled
Circuit-1	Joystick-Uno-L298N wiring
Image-3	Inside the updateJoystick() function
Code-1	determineXY_Output(), Joystick Setup

Image-4	The L298N module
Code-2	setup(), L298N Setup
Image-5	Octant regions and expected motor movement
Image-6	Motor Movement Checklist, blank
Image-7	Motor Movement Checklist, completed by the author
Image-8a	Variable bench power supply with variac
Circuit-2	Variable bench power supply, schematic (Image-8a, bottom panel)
Image-8b	Variable bench power supply, safety precautions
Table-1	Manifest
Table-2	Uno pin assignments
Table-3	Flags encoded by Bits()
Table-4	Troubleshooting

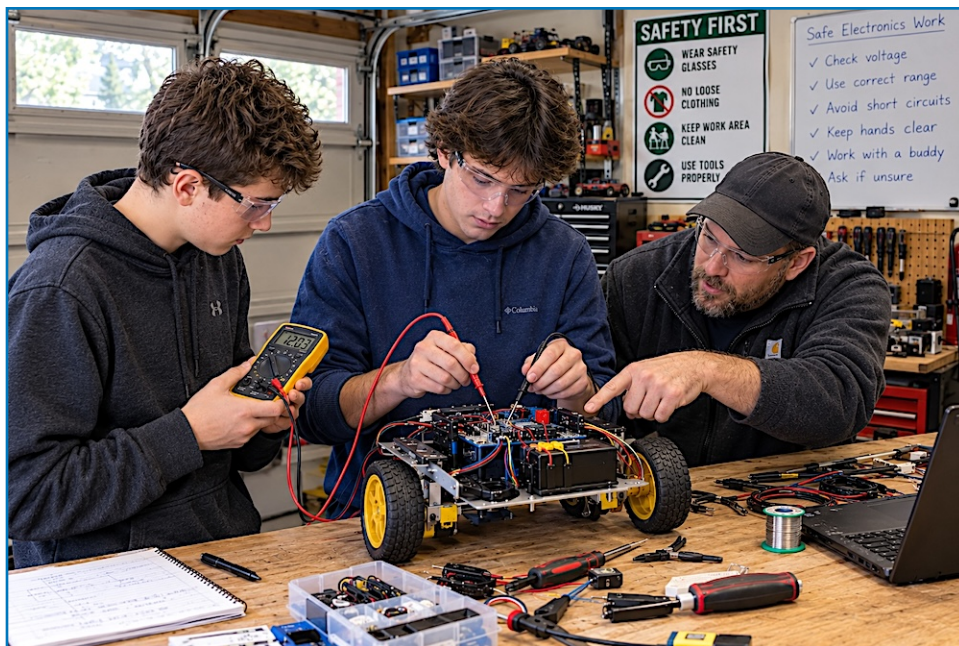
Purpose of This Guide

This guide is written for two readers: the adult who supervises a student through Articles 1000 to 1004 of the STEM Starter Kit Series, whether a parent, a teacher, or a mentor, and the student who works beside that adult. The articles teach the student. This guide states, for each article, what the student should learn, which terms the student should acquire, what the supervisor should verify at the bench, and which difficulties are most common. The supervisor uses it to plan and check each session; the student uses it as a record of what must be understood before the next article.

The series builds one project: a joystick driven pair of geared DC motors, controlled by an Arduino Uno through an L298N motor driver. The software for the project is kept in the Joystick-Uno-L298N repository of the MageMCU organization on GitHub. [1] A repository is a folder of source code and documents kept under version control, a system that records every change to every file. The guide exists so that the student learns to correct the behavior of the project in software and not by repeated rewiring, a practice that often ends in damaged parts and a frustrated student (see Programming Over Wiring).

The guide opens with safety, materials, and the Electronics Lab bench, and then has three sections. Article Summaries describes each article in order and the signal chain. Setup Procedures is the core of the guide, and its two setup procedures are the ones that the author has found most necessary for supervised students. Joystick Setup corrects the joystick axes in software with Code-1. L298N Setup corrects the motor directions in software with one value of the BitsL298N

enumeration in Code-2, and its Motor Movement Checklist, Image-5 to Image-7, checks each value until the correct one is found. Each fault is isolated before the next component is added. Troubleshooting lists common symptoms and their causes. An appendix describes the variable bench power supply with a variac, how the supervising adult uses it, and the advanced questions that it answers. A procedure in this guide is a numbered sequence of steps, and each is named by its purpose: the two setup procedures, Joystick Setup and L298N Setup; the checklist procedure of the Motor Movement Checklist; the measurement procedure, Measuring the Motor Bus and the L298N Drop; and, in the Appendix, the variac procedure, Using the Variac Supply.



The first engineering lab is often the family garage.

The Series at a Glance

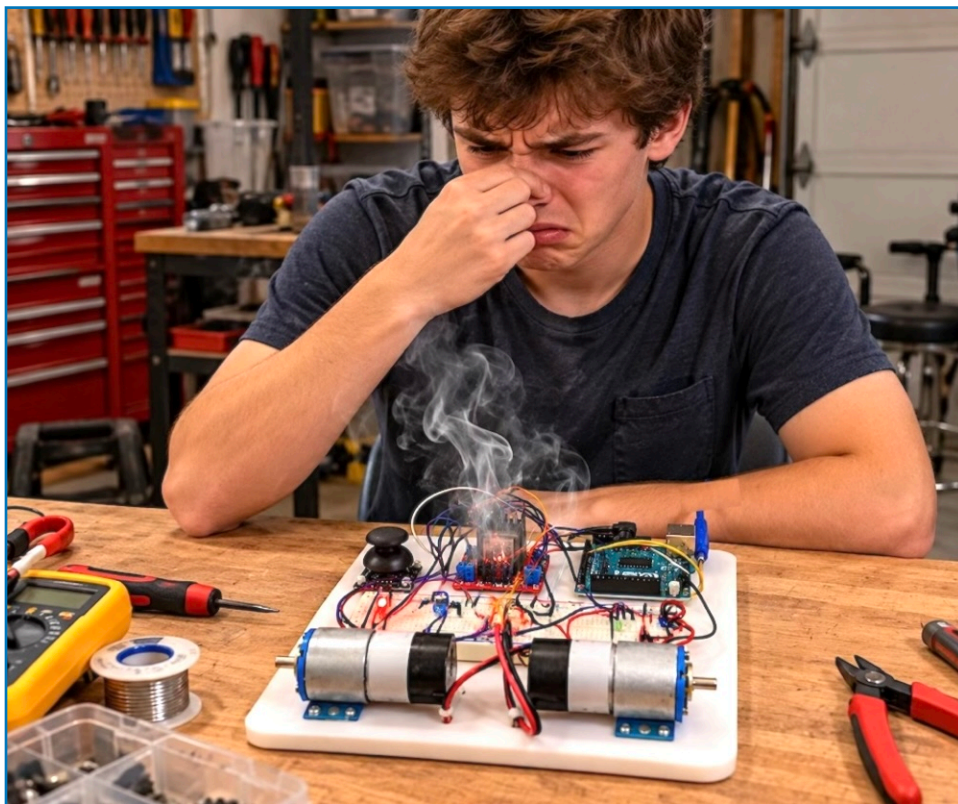
The guide covers Articles 1000 through 1004, Parts 1 through 5 of the series, and each article assumes the articles before it: Introduction Robotics (1000), Joystick Algorithm (1001), Arduino Uno: Pins, Ports, and Peripherals (1002), L298N Motor Driver (1003), and Experiments for Joystick-Uno-L298N (1004). Each article is summarized under Article Summaries.

Safety and Supervision

The student works with mains powered equipment, a motor supply capable of several amperes, and components that become hot. The supervisor is responsible for the following conditions at every session.

1. The student never works with electricity alone. A supervisor is present whenever a supply is connected.
2. The bench power supply is enclosed. Article 1000 describes the linear supply and the difference between the variac, a variable autotransformer that is not isolated from the AC line, and the step down transformer, which provides galvanic isolation. Galvanic isolation means that two circuits are coupled only through a magnetic field, with no conductive path between them. The student works only on the isolated low voltage side.
3. All grounds are joined. A common ground is a single reference conductor shared by the Uno, the logic side of the L298N, the motor supply, and the joystick. Without it, the signals have no common reference.
4. The motor supply is applied last and removed first. The L298N datasheet requires that the enable inputs be LOW before the motor supply is turned on and before it is turned off.[2] The L298N class does this at startup with `PinsL298N()` and on demand with `PowerDownL298N()`.
5. The heat sink of the L298N and the 7805 regulator become hot during operation and are not touched while the circuit is powered.
6. A spinning motor is not reversed abruptly. Article 1003 shows that reversal can drive the current far above the 2 A rating of the L298N; the command is reduced through zero before the direction changes.
7. Flammable material is kept away from the bench, and horseplay at the bench ends the session.
8. GitHub requires that each account holder be at least 13 years of age. A student younger than 13 uses the repository through the account of the supervising adult.

Programming Over Wiring



When the motors misbehave, change the program, not the wires.

He spent the afternoon on it. The motors turned the wrong way, so he moved a wire. They still turned the wrong way, so he moved another. Each attempt made him more impatient, and on the last one, he did not disconnect power or check the connection against Circuit-1. He pushed the wire into the wrong pin, and the L298N began to smoke. Now he sits alone at the bench, holding his nose, frustrated and disappointed, with power still connected. The wiring was never the problem. The motors turned the wrong way because the Bits() setting in his program did not match his circuit, and one change to that line, uploaded from his computer, would have corrected it. A mentor at his side would have seen the impatience, stopped him before the last wire, and turned him back to the program. Many students have been in this seat, and the lesson is the same for each of them: when the motors misbehave, change the program, not the wires, and never work alone.

Wire Trap. The wire trap is the cycle a student falls into when the student does not know there is a better way: the student moves wires to correct a wrong motor output, and each moved wire is a possible mishap, such as a wrong connection, a short circuit, or an open connection.

Wire Once. To avoid the wire trap, the student chooses a configuration, such as the joystick in any orientation and the motors in any order, then wires it one time, checks each connection, and fastens the wiring in place.

Programming Over Wiring. With the wiring fixed in place, the student corrects the joystick axes in Joystick Setup and the motor directions in L298N Setup. The motor directions are corrected with one value of the BitsL298N enumeration, sixteen four bit values from bits_0000 to bits_1111, chosen to match the program to the motor wiring.

Materials

Table-1 lists the parts and tools for Articles 1000 to 1004. A reader's parts may differ from the author's bench; the note for each part gives the rating that matters.

Manifest

Table-1

Item	Quantity	Articles Used	Note
Arduino Uno R3 and USB cable	1	1000 to 1004	Powered from its USB cable in Circuit-1
Thumb joystick module, five pin (GND, 5V, VRx, VRy, SW)	1	1001, 1004	Two 10 k Ω potentiometers and a push button; the push button circuit differs between modules and is tested as a black box with the Button class (Joystick Setup, step 4)
L298N motor driver breakout board	1	1003, 1004	Remove the ENA, ENB, and 5 V regulator jumpers
Geared DC motor with encoder	2	1003, 1004	Any rating the motor bus can supply, with a stall current within the 2 A per channel rating of the L298N. Stall current is the current drawn when the shaft is held stopped. The author's bench uses the Cytron SPG30-150K, 12 V, 26 RPM, 588 mN·m; the motor labels in Image-1 carry these values. A higher gear ratio gives a lower speed, a higher torque, and a higher encoder count per revolution of the output shaft.
7805 5 V regulator	1	1000, 1004	Logic supply; derives the 5 V DC bus from the motor bus
Step down transformer, enclosed	1	1000, 1004	Secondary chosen by motor rating (The Electronics Lab); the author's bench uses 18 VAC, 2 A to 3 A, 115/230 VAC primary, fully enclosed in a metal shield, three wire secondary with center tap (center tap not used)
Full wave bridge rectifier	1	1000, 1004	Produces the DC motor bus
Filter capacitor, 4700 μ F, 63 V, electrolytic	1	1000, 1004	Smooths the motor bus; observe polarity
Bleed resistor, 4.7 k Ω , 1 W	1	1000, 1004	Discharges the filter capacitor at power off
Line cord, three wire, grounded	1	1000, 1004	Earth ground bonded to the metal shield of the transformer
Fuse and power switch	1 each	1000, 1004	Fuse and switch the hot conductor ahead of the transformer primary
7805 capacitors, electrolytic and ceramic	1 each	1000, 1004	Output capacitor and bypass capacitor on the 5 V bus
Heat sink for the 7805	1	1004	Required when the motor bus is well above 12 V (Appendix)
Variac, 0 to 120 VAC	1 (optional)	1000, 1004	Variac supply only (Appendix); used by the supervising adult
Breadboard, jumper wires, resistors, LEDs, push buttons	1 set	1000 to 1004	Includes the indicator LED and its current limiting resistor (D12, or D3 in the advanced wiring; Table-2), and a 10 k Ω pull down resistor for the joystick SW pin, only if the module connects SW to 5 V
Polyethylene cutting board	1 or more	1000 to 1004	Base panel; drills cleanly and does not conduct. Parts are held with small cable ties through drilled holes.
Small cable ties	1 pack	1000 to 1004	Fasten each part to the base panel
Digital multimeter	1	1000 to 1004	Measures the motor bus and the drop of the L298N (Article 1003, Verify)
Lab notebook	1	1000 to 1004	Bound, pages numbered

The Electronics Lab

Image-1 shows the author's earlier joystick board, built on a single polyethylene cutting board; on this board the author developed the practice of programming over wiring. Image-2a and Image-2b show the Electronics Lab as rebuilt for the Joystick-Uno-L298N project, and Circuit-1 shows its complete wiring. The same bench is used for every experiment in Article 1004.

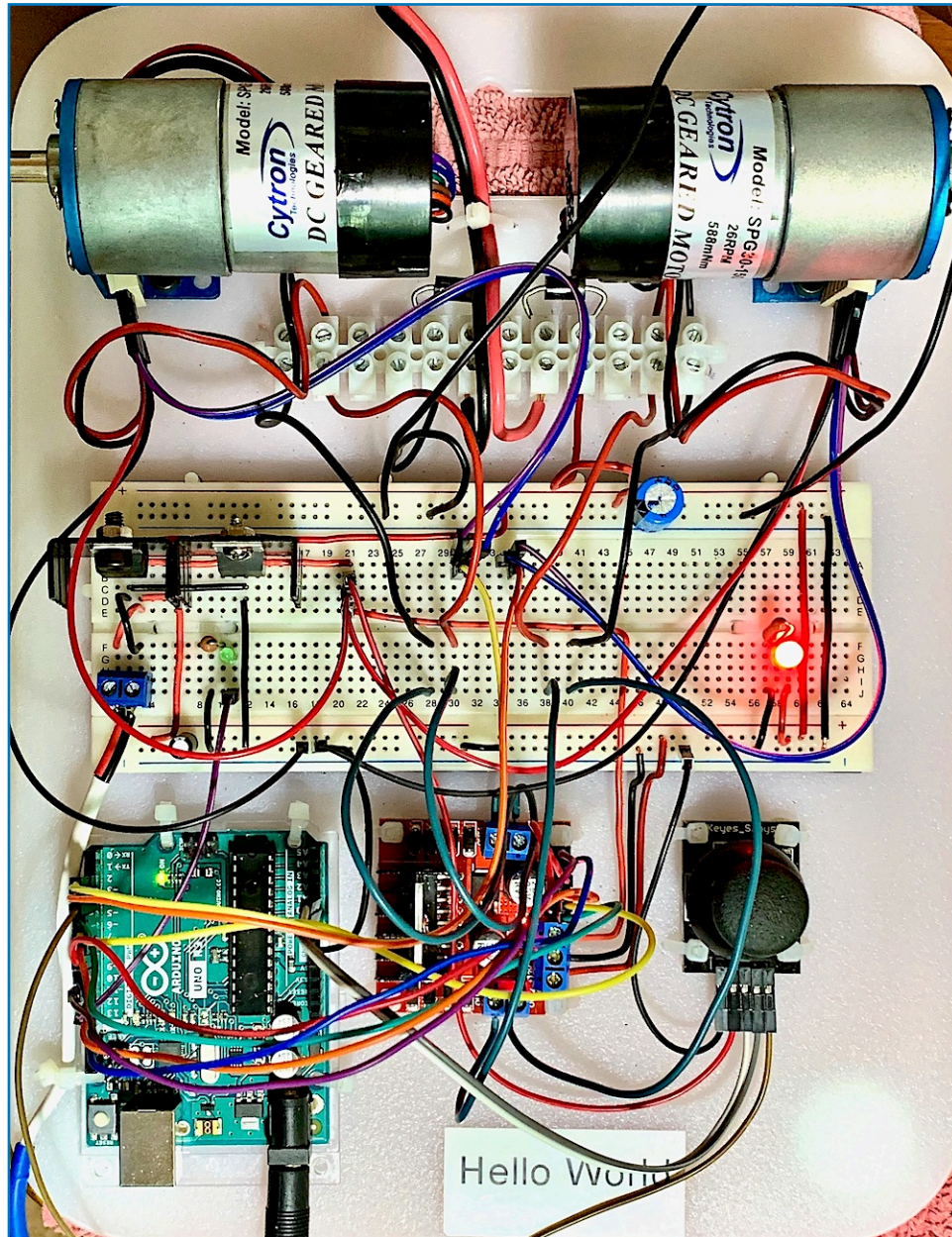
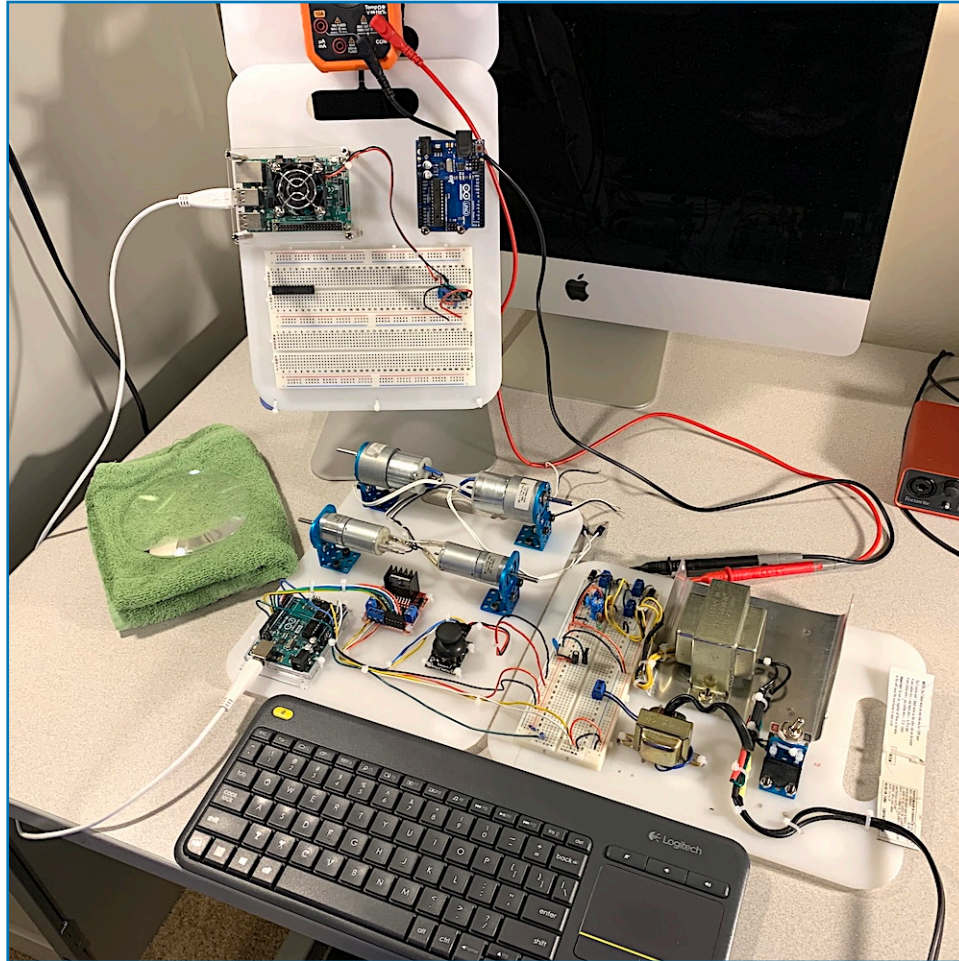


Image-1

The wire trap: on a board wired this densely, each moved wire is a possible mishap. Image-2a shows the same project wired once, with every correction made in the program.

Image-2a



The author's Electronics Lab for the Joystick-Uno-L298N project, on two polyethylene cutting boards with each part held by small cable ties. The left board holds the Uno, the L298N, the thumb joystick, and four 12 V geared motors. The right board holds the power supply: the 18 VAC transformer in its metal shield, the rectifier and regulator breadboard, and the power switch. The multimeter leads are on the motor bus, because the bus is measured before the motors are connected. The upper panel is the simple Uno setup from Article 1000: a Raspberry Pi 5 development computer with a second Uno and a breadboard. Article 1000 explains why the author uses the Raspberry Pi 5. Any computer that runs the Arduino toolchain, including the one behind the bench, can take its place. Compare this bench with Image-1. Every part is fastened down and the wiring is fixed, and the keyboard sits in front of the bench because every correction from here on is made in the program.

Power Supply

The motor supply is a linear supply built from a step down transformer, a full wave bridge rectifier, and a filter capacitor. The full wave bridge rectifier, four diodes arranged so that both halves of the AC cycle drive current in one direction through the load, converts the transformer secondary voltage into pulsating DC. The filter

capacitor, an electrolytic capacitor that charges to nearly the peak of each pulse, smooths the pulsating DC into the DC motor bus that supplies the motor terminal (VS) of the L298N. A 4.7 k Ω bleed resistor across the filter capacitor discharges it when the supply is turned off. The 7805 regulator derives the 5 V DC bus from the motor bus, and the 5 V bus supplies the logic terminal of the L298N and the joystick. The Uno is powered from its USB cable, and its GND pin is joined to the common ground of both buses.

Choosing the Motor Bus Voltage

The motor bus voltage is set by the rating of the motors. The output stage of the L298N drops approximately 2 V at 1 A, and the datasheet gives 1.8 V to 3.2 V.[2] The bus is therefore set about 2 V above the motor rating: 14 V for 12 V motors. The filtered bus is approximately $1.414 \times V_{\text{secondary}} - 1.4$ V, where 1.414 converts the RMS secondary voltage to its peak and 1.4 V is the drop across the two conducting diodes of the bridge. RMS (root mean square) is the value that a meter reads for AC. The drop varies with the current and between individual modules; at 2 A the datasheet gives a total drop of up to 4.9 V.[2] The reader therefore measures it with the measurement procedure in Measuring the Motor Bus and the L298N Drop, below; Article 1003, Verify, introduces the same measurement. Either of two supplies produces the bus:

1. Fixed supply. The transformer has one secondary voltage, chosen as $(V_{\text{bus}} + 1.4) / 1.414$. For a 14 V bus the secondary is $15.4 / 1.414 = 10.9$ VAC; the nearest common transformer, 12 VAC, gives $12 \times 1.414 - 1.4 = 15.6$ V unloaded. The fixed supply suits a reader who has chosen the motors and will not change them.
2. Variac supply. The supply of the Appendix, Circuit-2, in which a variac sets the transformer primary voltage. The supervising adult raises the variac slowly while the bus is measured with a multimeter, and stops when the bus reads the required voltage. With the 4:1 transformer of Circuit-2, a 14 V bus needs a secondary of 10.9 VAC, which is a variac setting of about $4 \times 10.9 = 44$ VAC. The variac supply suits a reader who has not yet chosen motors or who tests motors of several ratings.

Under load, the bus falls below its unloaded value by up to the ripple voltage, the drop of the output between the charging pulses of the rectifier, about 1.8 V at 1 A with a 4700 μF capacitor (see the Appendix), so the final bus measurement is made with the motors running. The 7805 requires about 7 V at its input at all times, and the ripple takes about 2 V more, so a bus below about 9 V, as for 6 V motors, cannot supply the 5 V bus. In that case the joystick and the logic terminal of the L298N take 5 V from the 5V pin of the Uno. Appendix, Advanced Questions with the

Variable Supply, examines the fall of the bus under load and the input limit of the 7805.

Measuring the Motor Bus and the L298N Drop

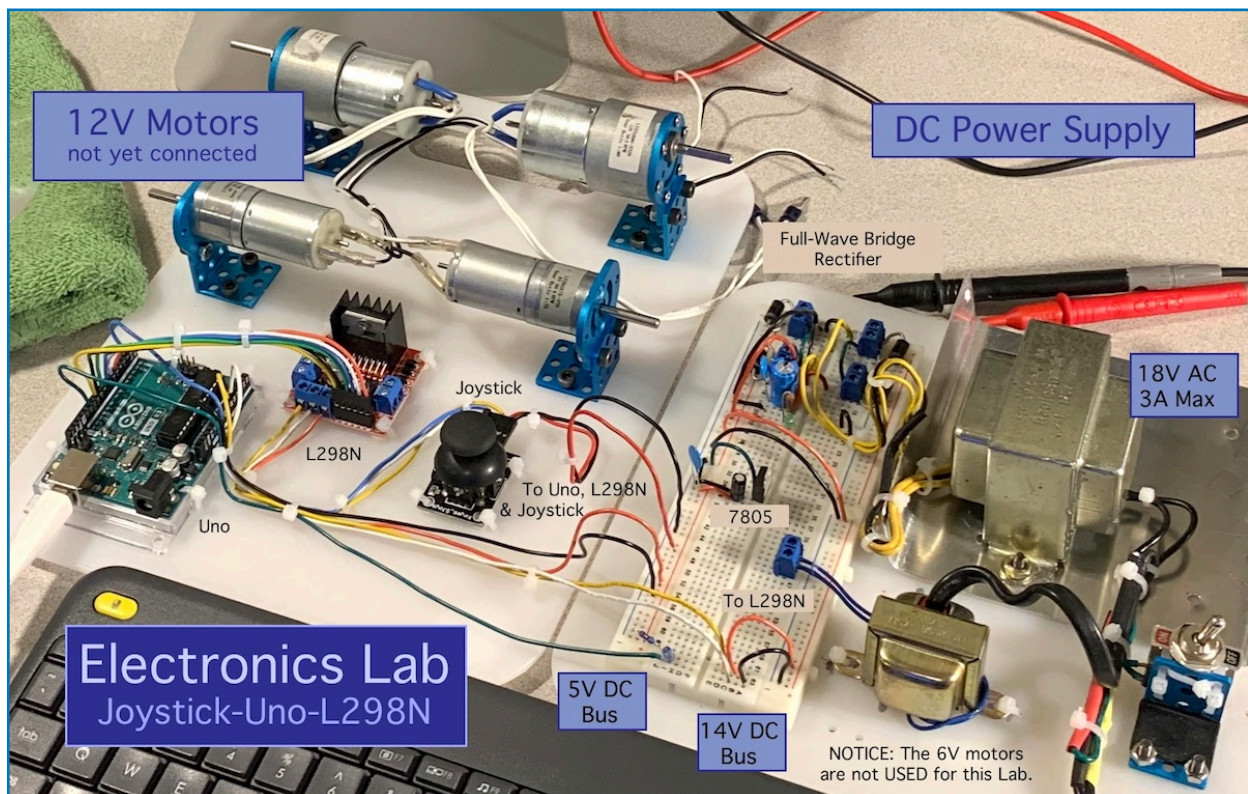
The bus voltage written on any label, including the 14V DC label in Image-2b, is a design value. The reader measures the bus with the multimeter, and records four values: the bus with no load, the bus under load, the motor voltage, and the drop of the L298N. The supervising adult applies and removes power; the student reads the multimeter and records each value in the notebook. This measurement procedure is performed in L298N Setup, step 8, again after the Motor Movement Checklist, and whenever the supply or the motors are changed.

1. Set the multimeter to DC volts, on a range above the expected bus voltage.
2. Measure the no load bus. With the motors stopped, measure from the VS terminal of the L298N to GND. Without load current, the output stage of the L298N conducts no current, so this reading is close to the unloaded value, $1.414 \times V_{\text{secondary}} - 1.4$ V. Record it.
3. Run the motors at full duty. Hold the joystick fully forward, so that both commands are 255. At 255 the enable output is continuously HIGH, so the multimeter reads a steady DC value instead of a PWM average.
4. Measure the loaded bus. With the motors running, measure from VS to GND again. Record it. The difference between steps 2 and 4 is the fall of the bus under load.
5. Measure the motor voltage. With the motors running, measure across the two output terminals of one channel, OUT1 and OUT2. Record it.
6. Calculate the drop. The drop of the L298N is the loaded bus minus the motor voltage. Record it, and compare it with the datasheet: 1.8 V to 3.2 V at 1 A, and up to 4.9 V at 2 A.[2]
7. Adjust or confirm. With the variac supply, raise the bus by the measured drop so that the motor voltage reaches, but does not exceed, the motor rating, then repeat steps 2 to 6. With the fixed supply, confirm that the motor voltage at full duty does not exceed the motor rating.
8. Release the joystick and stop the motors. The supervisor signs the entry. A drop well above the datasheet range, or a heat sink that becomes very hot, points to a high motor current (Table-4).

The Author's Bench

The author's bench in Image-2b uses 12 V motors and an 18 VAC transformer. The blue electrolytic capacitor across the rectifier output is the filter capacitor of the motor bus, and it sits ahead of the 7805. On the 5 V bus, a small electrolytic capacitor and a ceramic capacitor serve the 7805. The electrolytic output capacitor supplies brief surges of load current and keeps the regulator stable; the ceramic capacitor is a bypass capacitor, which gives high frequency noise a short path to ground.

Image-2b



The Electronics Lab with each section labeled. Power flows from right to left: the 18 VAC transformer, the full wave bridge rectifier, the filter capacitor, the DC motor bus to the L298N, and the 7805 regulator, which supplies the 5 V DC bus to the logic of the L298N and the joystick. The 14V DC Bus label and the motors shown are explained in the paragraphs that follow.

Filtered, 18 VAC gives $18 \times 1.414 - 1.4 = 24$ V unloaded, which suits motors rated near 22 V to 24 V, not the 12 V motors of the bench. The 14V DC label in Image-2b is therefore the design value of the bus for 12 V motors, the motor rating plus about 2 V for the drop of the L298N, and not the output of the 18 VAC transformer.

The photograph was taken while the unloaded bus was being measured, before the motors were connected. In a later measurement with the bus set to 15 V, the measurement procedure above gave about 12.8 V at the motor terminals with the motors running, a drop of about 2.2 V, which is within the 1.8 V to 3.2 V of the datasheet.[2] The motors and the heat sink of the L298N stayed cool. The dissipation of the L298N is the drop times the motor current, so the heat sink warms under heavy load or stall, when the current approaches 2 A. Appendix, Advanced Questions with the Variable Supply, item 2, examines the drop at other currents.

The reader does not rely on any label. The bus is measured with the measurement procedure in Measuring the Motor Bus and the L298N Drop, and every reading is recorded in the notebook. A reader with 12 V motors uses a 12 VAC transformer or the variac supply. Image-2b shows four 12 V motors, not yet connected; the project uses two. The small transformer below the 18 VAC transformer and the 6 V motors named in the notice of Image-2b are not used in this project.

Wiring

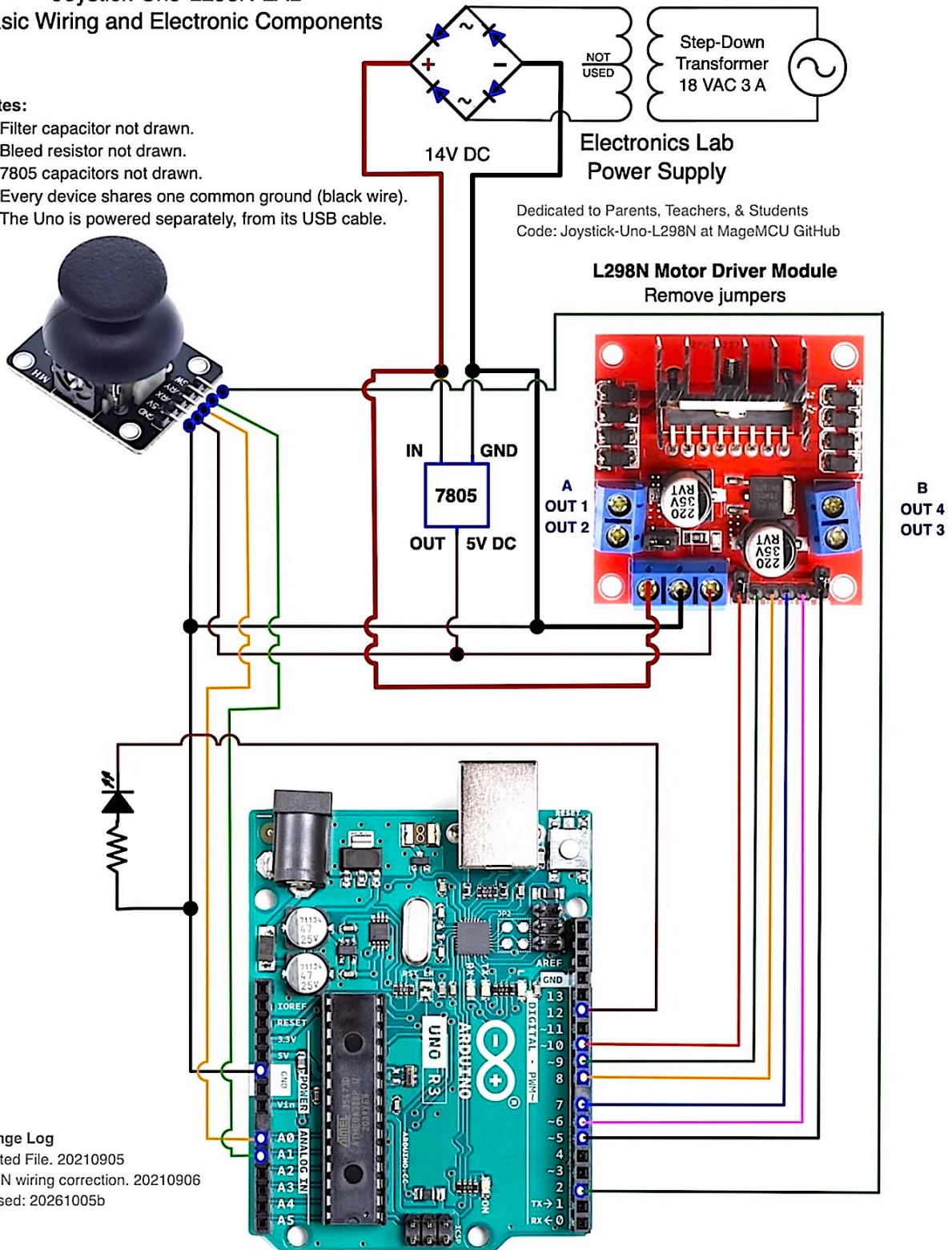
Circuit-1 shows every connection between the power supply, the joystick, the indicator LED, the Uno, and the L298N. All three jumpers of the L298N are removed. As its notes state, Circuit-1 does not draw the filter capacitor and the bleed resistor across the rectifier output or the two 7805 capacitors (The Author's Bench), and it does not draw a resistor on the joystick SW pin (D2). A pull up resistor holds an input HIGH until a switch connects it to ground; without a pull up or a pull down resistor, D2 is a floating input, an input connected to nothing, which reads HIGH or LOW at random. With `activeLow = true`, the Button class sets D2 to the INPUT_PULLUP mode, which connects the internal pull up resistor of the ATmega328P, a resistor of about 20 k Ω to 50 k Ω inside the chip, so D2 does not float and no external pull up resistor is needed.[3] Thumb joystick modules differ, and the reader treats the module as a black box, as in Article 1000: its circuit is not known, so it is first tested with the Button class as supplied, in Joystick Setup, step 4, and its circuit is measured only if that test fails. With `activeLow = true`, the default, the Button class reads a push button that connects SW to GND: HIGH when released and LOW when pressed. A module whose push button connects SW to 5 V needs an external 10 k Ω pull down resistor from SW to GND, and the Button object is then created with `activeLow = false`.

Joystick-Uno-L298N LAB

Basic Wiring and Electronic Components

Notes:

- (1) Filter capacitor not drawn.
- (2) Bleed resistor not drawn.
- (3) 7805 capacitors not drawn.
- (4) Every device shares one common ground (black wire).
- (5) The Uno is powered separately, from its USB cable.



Change Log

Created File. 20210905
 L298N wiring correction. 20210906
 Revised: 20261005b

Jesse Carpenter - Carpenter Software (carpentersoftware.com) - ALL RIGHTS RESERVED

Article Summaries

Article 1000, Introduction Robotics (Part 1)

Purpose. The article introduces the scientific method through the black box, a system whose internal workings are unknown and must be learned by measurement, and it establishes the lab notebook as the permanent record of every experiment. It then describes the differential drive wheeled mobile robot (DWMR), the workspace, each component of the kit, the power supply, the development computer, the first program (Blink), and the multimeter.

Vocabulary. Black box, scientific method, lab notebook, corroborating witness, differential drive wheeled mobile robot (DWMR), microcontroller unit (MCU), quadrature encoder, galvanic isolation, variac, common ground, toolchain, Ohm's law.

Verify. The student keeps dated notebook entries, and the supervisor signs them as the corroborating witness, the person who confirms that the work occurred on the recorded date. The Blink program compiles and runs on the Uno. The student measures one resistor and one voltage with the multimeter and confirms the result with Ohm's law ($V = I \times R$).

Common difficulties. The joystick wiring in 1000 places X on A0 and Y on A1, while the repository places X on A1 and Y on A0. The Joystick Setup procedure in this guide resolves the difference by measurement, and the student should record which assignment the bench uses.

Article 1001, Joystick Algorithm (Part 2)

Purpose. The article explains how the two potentiometers of the thumb joystick become two numbers in the Uno, and how the joystick algorithm converts those numbers into a left and a right motor command. It develops the resistor circuits needed to understand the potentiometer, the linear map that converts one range into another, and the octant algorithm, which divides the joystick area into eight regions.

Vocabulary. Potentiometer, wiper, series and parallel resistance, equivalent resistance, voltage divider, Kirchhoff's voltage law (KVL), Kirchhoff's current law (KCL), analog to digital converter (ADC), quantization, resolution, linear interpolation, dead zone, octant.

Verify. The student calculates the output of a voltage divider and confirms it with the multimeter. The student maps an ADC reading from the range 0 to 1023 into the range -1 to 1 by hand and compares the result with the LinearMap output.

Common difficulties. Small readings near the joystick center cause the motors to creep or jitter. The dead zone, a small region around the center in which the output is forced to zero, removes this effect; its size is tuned per joystick in Article 1004.

Article 1002, Arduino Uno: Pins, Ports, and Peripherals (Part 3)

Purpose. The article describes the Arduino Uno R3 and the ATmega328P, the microcontroller on the board, one feature at a time: general purpose input and output (GPIO), the basic circuits for buttons and LEDs, pull up and pull down resistors, logic levels, the serial monitor, the peripherals (ADC, I2C, PWM, SPI, radio, and UART), and the use of the Uno as an in system programmer (ISP) for a bare ATmega328P on a breadboard.

Vocabulary. GPIO, port, pull up resistor, pull down resistor, floating input, current limiting resistor, logic level, peripheral, serial communication, in system programmer (ISP), fuse (a configuration setting stored in the ATmega328P, not a power fuse), lock bit, bootloader.

Verify. A push button with an external pull up resistor reads HIGH when released and LOW when pressed; with an external pull down resistor, as in Experiment-1 of Article 1004, it reads LOW when released and HIGH when pressed. The student calculates the current limiting resistor for an LED before connecting it.

Common difficulties. Fuse and lock bit settings are written into the ATmega328P during the ISP procedure. An incorrect fuse value, especially a clock source setting, can leave the chip unresponsive to further programming until the setting is corrected. The supervisor checks the values against the article before the student writes them.

Article 1003, L298N Motor Driver (Part 4)

Purpose. The article describes the L298N as a component: the H bridge, which reverses the voltage across a motor; the saturation voltage of the bipolar output stage; the board connections; pulse width modulation (PWM) and the timers of the Uno; the physics of the permanent magnet DC motor; the coast and brake states; and the L298N class.

Vocabulary. H bridge, bipolar junction transistor, Darlington transistor, saturation voltage, enable input, pulse width modulation (PWM), duty cycle, timer, prescaler, back electromotive force (back EMF), torque constant, flyback diode, coast, brake.

Verify. The three jumpers of the breakout board are removed. The student measures the supply voltage and the motor terminal voltage and calculates the drop across the L298N. The student predicts the average motor voltage from the duty cycle and confirms it with the multimeter. This guide gives the measurement as the measurement procedure under Measuring the Motor Bus and the L298N Drop. Appendix, Advanced Questions with the Variable Supply, continues this measurement with the variable supply.

Common difficulties. Article 1003 describes the coast and brake states and gives a worked example of braking current. Braking is an advanced technique, and the author has never used a brake routine in this project. The robot operates on a flat surface, and with a high gear ratio, such as that of the author's SPG30-150K motors, the friction of the gear train stops the robot soon after the command returns to zero, so the motors coast to a stop and no brake is needed. A supervisor who explores braking with an advanced student first confirms that the student understands why the braking current must stay below 2 A.

Article 1004, Experiments for Joystick-Uno-L298N (Part 5)

Purpose. Article 1004 is the build article. It wires the joystick and the L298N to the Uno, describes the Electronics Lab bench and its power supply, walks through the software in the repository, and presents the experiments that take the student from an unpowered breadboard to a joystick driven pair of motors.

Experiments, as outlined on 20260905 (incomplete, work in progress):

Experiment-1, Timing and Button Input on the Uno (three software labs: delay(), the Timer class, the Button class).

Experiment-2, Wiring and Reading the Thumb Joystick (this guide, Joystick Setup).

Experiment-3, L298N Single Motor.

Experiment-4, L298N Two Motors and the Bits Table (this guide, L298N Setup).

Experiment-5, Joystick to Motors (this guide, L298N Setup, Motor Movement Checklist).

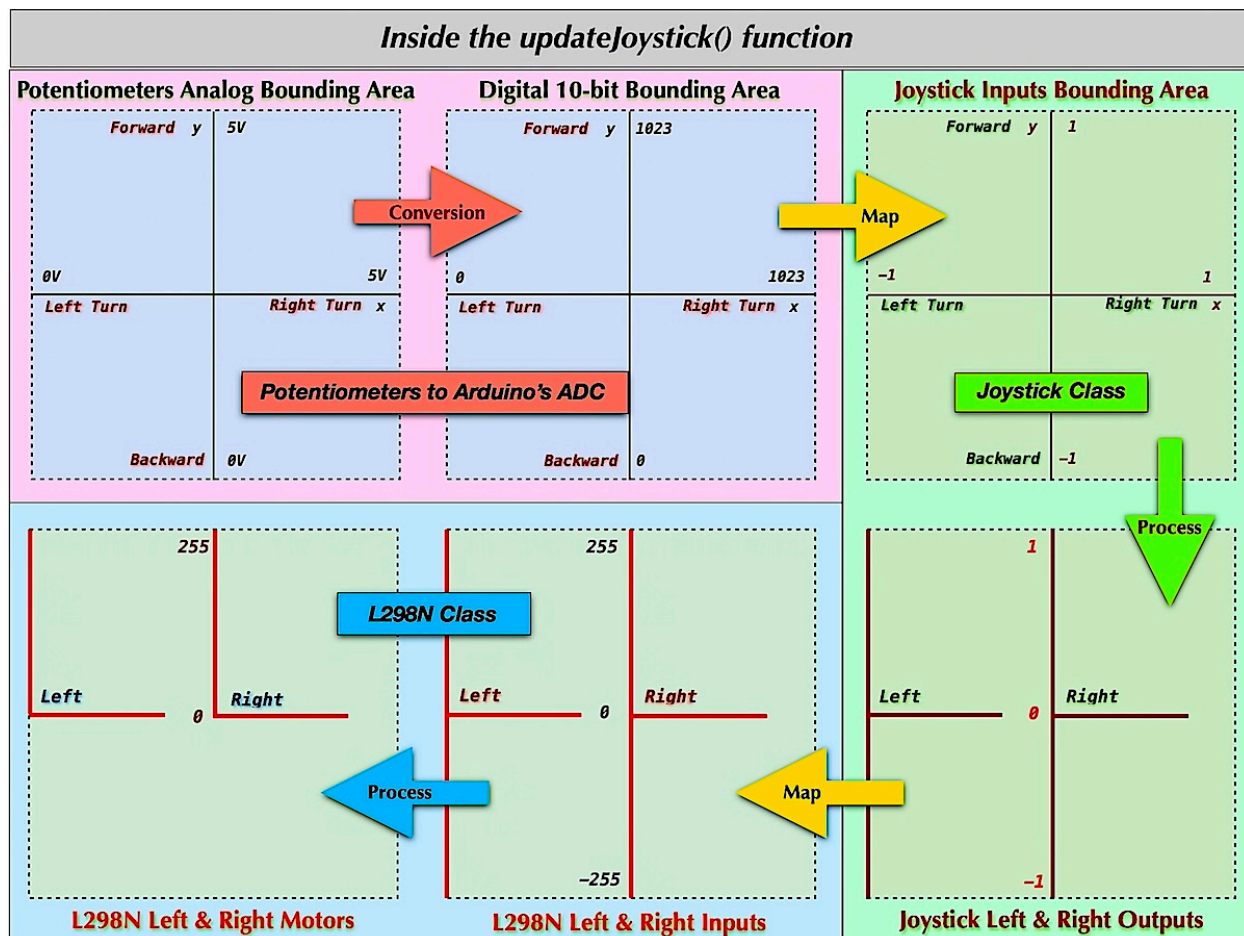
Vocabulary. Blocking call, a function call that does not return until its work is finished; nonblocking design, in which work is divided into short steps so that loop() continues to run; signal chain; debounce; latching and momentary button modes.

Verify. Each experiment ends with an expected output, and the student records the observed output in the notebook beside it.

The Signal Chain

Image-3 shows every stage between the joystick and the motors, as performed inside the `updateJoystick()` function. The two potentiometers produce voltages from 0 V to 5 V. The ADC of the Uno converts each voltage into an integer from 0 to 1023, which is a 10 bit value. The `LinearMap` class maps each integer into the range -1 to 1 . The `Joystick` class applies the octant algorithm of Article 1001 and produces a left and a right output, each in the range -1 to 1 . A second `LinearMap` maps each output into the range -255 to 255 , and the `L298N` class writes the sign to the direction inputs and the magnitude to the enable input as a PWM duty cycle.

Image-3



The signal chain inside `updateJoystick()`. The ADC converts each potentiometer voltage, 0 V to 5 V, into an integer from 0 to 1023. `LinearMap` maps each integer into the range -1 to 1 , the `Joystick` class produces the left and right outputs, a second `LinearMap` scales them to -255 to 255 , and the `L298N` class drives the motors.

Uno Pin Assignments

Table-2 lists every Uno pin used by the project. The second column is the physical connection drawn in Circuit-1. The third column is the name the software gives the pin: the joystick and button pins in the Step1_Joystick program (Code-1) and the Step2_JUL program (Code-2) and the default pins of the L298N class in Article 1003.

Table-2

Uno pins	Circuit-1 connection	Software name
A0	Joystick VRx	yDigital, analogRead(A0)
A1	Joystick VRy	xDigital, analogRead(A1)
D2	Joystick SW	buttonPin
D3	Indicator LED, advanced wiring (Article 1003, Table-4)	ledPin
D5	L298N ENB	_EN_A (LeftEN)
D6	L298N IN4	_LeftIN_A (LeftA)
D7	L298N IN3	_LeftIN_B (LeftB)
D8	L298N IN2	_RightIN_A (RightA)
D9	L298N IN1	_RightIN_B (RightB)
D10	L298N ENA	_EN_B (RightEN)
D12	Indicator LED	ledPin

Uno pin assignments. The Circuit-1 connection is the physical wire; the software name is the name used in Code-1, Code-2, and the L298N class.

The two columns differ, and the differences are the reason for the setup procedures that follow. At the joystick, the axis the software calls x is wired to VRy; whether that is correct depends on how the joystick is mounted, and Joystick Setup determines it by measurement. At the L298N, Circuit-1 connects the logic header in the reverse order of the class defaults: the software's left group (D5, D6, D7) drives channel B, and its right group (D8, D9, D10) drives channel A. Each enable pin still travels with its own IN pair, so no wire is moved; the Bits() setting corrects the assignment in software (Table-3).

Article 1003, Table-4, gives the advanced wiring, which places both enable inputs on Timer1 (D9 and D10). The advanced wiring is selected by passing the pins to the L298N constructor, and the setup procedures in this guide apply to either wiring.

Arduino pin numbers. The number that the software gives a pin is the Arduino pin number printed beside the header of the Uno, not the pin number of the ATmega328P package. The line `int ledPin = 3` selects Arduino digital pin 3, labeled 3 on the header and D3 in this guide, which is bit 3 of port D of the chip (PD3); the comment `// UNO D3 (CHIP-PD3)` in Code-2 gives both names. D12 is bit 4 of port B (PB4). The indicator LED uses D12 in the wiring of Circuit-1 and D3 in the advanced wiring of Article 1003, Table-4, where D12 becomes IN4. Code-2, line 42, still shows `ledPin = 3` from the earlier wiring, and that difference is a useful troubleshooting example. With the LED wired to D12 and `ledPin = 3`, the push button works but the LED never lights. The wiring is correct; the fault is one number in the program. The student places comment marks on line 42 and removes them from line 44, so that `ledPin` is 12, the pin that the LED is wired to, uploads the program, and confirms that the LED lights: programming over wiring on a small scale.

Setup Procedures

Joystick Setup and L298N Setup are the core of this guide. Joystick Setup corrects the joystick axes in software with Code-1. L298N Setup corrects the motor directions in software with one of the sixteen values of the `BitsL298N` enumeration in Code-2; each value placed in Code-2 is uploaded and checked with the Motor Movement Checklist, Image-5 to Image-7, until the correct value is found. The wiring is completed and checked once; every correction after that is made in the program.

Why Setup Is Done in Two Stages

The software must match the physical wiring in two places. At the joystick, three conditions can each be true or false: the two axes can be exchanged, the x axis can be reversed, and the y axis can be reversed. That gives $2 \times 2 \times 2 = 8$ possible joystick orientations. At the L298N, the `Bits()` method of the L298N class encodes four Boolean flags, which gives $2 \times 2 \times 2 \times 2 = 16$ possible patterns. If both are tested together, a student faces $8 \times 16 = 128$ combinations, and a wrong result at the motors cannot be traced to its cause.

The two setup procedures therefore separate the two stages. Joystick Setup fixes the eight joystick conditions by measurement at the serial monitor, with no motors connected. L298N Setup then fixes the sixteen `Bits()` patterns by observation of the motors, with the joystick already known to be correct. Each stage has one kind of fault and one kind of test.

Joystick Setup

Joystick Setup sets the axes and the directions of the joystick in software. It uses the `determineXY_Output()` function of the `Step1_Joystick` program, Code-1, and it is done with no mains voltage on the bench and no motors connected. The joystick may be mounted in any orientation; Joystick Setup corrects the orientation in software.

Code-1 reads the button state first (line 68). While the button state reads ON, it reads the two potentiometers (lines 82 and 84) and prints them on one line that begins "Analog:" (line 119), with the x reading after "a:" and the y reading after "b:", for example `Analog: a: 512 b: 509`. While the button state reads OFF, it prints "Button OFF" and no readings. Lines 83 and 85 are the reversed forms of lines 82 and 84; they are comments until a step below calls for them.

The push button is read by the `Button` class of the repository (`Button.h`). The `setup()` function of `Step1_Joystick` creates the `Button` object on line 51 with `Button(buttonPin, ledPin)`, and the `setup()` function of `Step2_JUL`, Code-2, creates it in the same way on line 45: `buttonPin` is 2 (D2), and `ledPin` is the indicator LED. The two arguments not written take their default values: `activeLow = true`, for a button that reads LOW when pressed, and a debounce time of 50 ms. Debounce is the rejection of the rapid, unwanted transitions that a mechanical contact produces when it closes or opens. The class is latching by default: each press toggles the button state between ON and OFF, and the state stays after the button is released. Code-1 asks the object for that state on line 68 with `isButtonOn()`. Joystick Setup uses the `Button` class as supplied; step 4 tests the push button with it.

1. Prepare the bench. The line cord of the bench power supply stays unplugged, so no mains voltage is present on the bench, and the L298N and the motors are not connected. The Uno is powered by its USB cable only.
2. Wire the joystick. Connect VRx, VRy, and SW to the Uno pins listed in Table-2, as drawn in Circuit-1, and take 5 V and GND from the 5V and GND pins of the Uno.
3. Upload. Open the Code-JUL folder of the repository. In `platformio.ini`, line 31, select the `Step1_Joystick` program:

```
build_src_filter = +<*> +<Step1_Joystick/> -<Step2_JUL/>
```

In `Common.h`, remove the comment marks from line 33, so that it reads `#define DEBUG_MAIN`. Upload the program and open the serial monitor at 9600 baud. While any debug flag is defined, the program reads the button state and prints once every 3 s (`BUTTON_TIMER_mS` in `Common.h`).

```

64 // STEP1: Setup Joystick-UNO
65 void determineXY_Output()
66 {
67     //
68     bool buttonFlag = buttonDebug.isButtonOn();
69
70 #ifdef DEBUG_MAIN
71     if (buttonFlag)
72         Debug("Button ON");
73     else
74         Debug("Button OFF");
75 #endif
76
77     //
78     if (buttonFlag)
79     {
80         // Analog to Digital (10-bit) Conversion
81         // Digital Values from 0 to 1023
82         int xDigital = analogRead(A1);
83         // int xDigital = 1023 - analogRead(A1);
84         int yDigital = analogRead(A0);
85         // int yDigital = 1023 - analogRead(A0);
86         // -----
87         // See articles at Carpenter Software (carpentersoftware.com)
88         // especially, 1009 Parent, Teacher, and Student Guide, in the
89         // section Joystick Setup.
90         // In order for the Joystick Algorithm to
91         // work properly, the following values has to occur:
92         //
93         //          (y-axis)
94         //          FORWARD | 1023
95         //          |
96         //          |
97         // 0          (511, 511)          1023
98         // ----- (x-axis)
99         // LEFT TURN | RIGHT TURN
100        //          |
101        //          |
102        //          |
103        //          BACKWARD | 0
104        // -----
105        //
106        // MOVE the joystick as shown on the graph.
107        // Test x-axis values changes
108        // Test y-axis values changes
109        // Next,
110        // Test x-axis from left 0 to right 1023
111        // Test y-axis from backward (down) 0 to forward (up) 1023
112        // If the values are reversed, for example, 1023 to 0,
113        // use the following equation tested for each:
114        // xDigital = 1023 - analogRead(A1); // could be A0
115        // yDigital = 1023 - analogRead(A0); // could be A1
116        //
117        // Debug the output... (See MiscMath.h for Debug() functions)
118 #ifdef DEBUG_MAIN
119     Debug<int>("Analog: ", xDigital, yDigital);
120 #endif
121     }
122 }
123

```

4. Test the push button as a black box. Thumb joystick modules from different manufacturers do not have the same circuit, so the student treats the module as a black box, as in Article 1000: the module is tested with the Button class as supplied before any attempt is made to learn its circuit. Without tilting the joystick grip, the knob that the thumb moves, press and release the push button several times, and after each press wait for the next line on the serial monitor, up to 3 s, before pressing again. Each press must change the serial monitor from "Button OFF" to "Button ON", or back, exactly once, and the state must not change while the button is untouched. If the module passes this test, it works with the Button class; record "push button passes" in the notebook and go to step 5. If the state changes by itself, changes more than once per press, or never changes, disconnect the USB cable and SW, power the Uno again, and measure from the SW pin of the module to GND with the multimeter, released and pressed:

- About 5 V released and 0 V pressed, or about 0 V in both positions: the button connects SW to GND. Use `activeLow = true`, the default, and check the SW connection to D2.
- About 0 V released and 5 V pressed: the button connects SW to 5 V. Add a 10 k Ω pull down resistor from SW to GND, and create the Button object with `activeLow = false` in both programs, `Button(buttonPin, ledPin, false)`: line 51 of `Step1_Joystick` and line 45 of `Code-2`.
- Record the result in the notebook, reconnect SW to D2, upload, and repeat this step.

5. Turn on the readings. Press and release the push button until the serial monitor prints "Button ON" and the Analogs line appears. The Button class is latching by default: each press toggles the button state between ON and OFF.

6. Check the x axis. Move the joystick left and right. The x value must change, and the y value must stay near its center reading of about 511. If the y value changes instead, the axes are exchanged; exchange A0 and A1 in lines 82 and 84, upload, and repeat this step.

7. Check the x direction. Move the joystick fully left; the x value must approach 0. Move it fully right; the x value must approach 1023. If the values are reversed, place comment marks on line 82 and remove them from line 83, so that the reading is subtracted from 1023, then upload and repeat this step:

```
int xDigital = 1023 - analogRead(A1); // could be A0
```

8. Check the y direction. Move the joystick fully down (backward); the y value must approach 0. Move it fully up (forward); the y value must approach 1023. If the values are reversed, place comment marks on line 84 and remove them from line 85, then upload and repeat this step:

```
int yDigital = 1023 - analogRead(A0); // could be A1
```

9. Record. The student records in the notebook the pin assigned to each axis, whether each axis is reversed, and the two readings at rest. Readings at rest near 511 are expected; small offsets from center are normal and are removed by the dead zone. The supervisor signs the entry.

10. Close Joystick Setup. Press and release the push button once, so that the serial monitor prints "Button OFF". From this point, the joystick wiring is not changed again, and the `analogRead()` lines change only once more, when L298N Setup, step 5, copies them into `Step2_JUL`. The `determineXY_Output()` function is used only for this setup and for the check in L298N Setup, step 7.

L298N Setup

L298N Setup connects the L298N and the motors and selects the value of the `BitsL298N` enumeration that matches the program to the motor wiring. An enumeration is a C++ type whose values are a fixed list of names; `BitsL298N` has sixteen values, `bits_0000` to `bits_1111`, and each value is four binary digits, one for each flag in Table-3. Image-4 shows the L298N module: the Motor A and Motor B outputs, the power terminals, and the ENA, IN1, IN2, IN3, IN4, and ENB inputs.

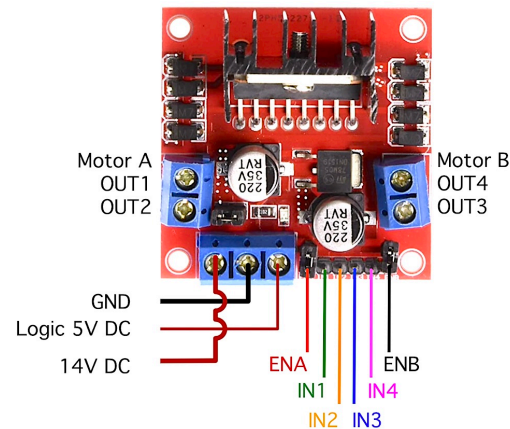
Image-4

```
analogWrite(EN, PWM) where EN is bit 3  
and PWM is bit 2.
```

```
analogWrite(ENA, absT<int>(_pwmA));  
analogWrite(ENB, absT<int>(_pwmB));
```

```
digitalWrite(_LeftIN_A, LOW) where  
LeftIN is bit 1 and RightIN is bit 0.
```

```
digitalWrite(_LeftIN_A, LOW);  
digitalWrite(_LeftIN_B, HIGH);
```



The L298N module as wired in Circuit-1, with two fragments of the L298N class. The upper fragment drives the enable inputs, ENA and ENB, with `analogWrite()`, so the magnitude of each command becomes a PWM duty cycle; bits 3 and 2 of the `BitsL298N` value select which enable pin and which command are paired. The lower fragment drives one IN pair with `digitalWrite()`, one pin LOW and the other HIGH, so the sign of the command sets the direction of the motor; bits 1 and 0 select the direction convention of the left and right IN pairs.

The `setup()` function, Code-2, creates the L298N object with the six pin numbers (lines 55 to 71), sets the enable inputs LOW with `PinsL298N()` (line 73), and selects one value of the enumeration with `Bits()` (line 92). Line 92 is the only line changed during the Motor Movement Checklist. The comments in Code-2 predate the current L298N.h; Table-3 gives the current meaning of each bit. The variables ENA, IN1, IN2, IN3, IN4, and ENB in Code-2 name the Uno pins D5 to D10 in the order of the class defaults; they are not the silkscreen labels of the L298N, which Circuit-1 connects in the reverse order (Table-2).

1. Power down. Disconnect the USB cable and unplug the line cord of the bench supply. Measure 0 V across the filter capacitor before touching the circuit.
2. Remove the jumpers. Remove the ENA, ENB, and 5 V regulator jumpers of the L298N.
3. Wire the L298N. Connect D5 to D10 of the Uno to the logic header of the L298N as listed in Table-2 and drawn in Circuit-1. Connect the Motor A and Motor B outputs to the two motors, the VS terminal to the motor bus, the logic terminal to the 5 V bus, and GND to the common ground. Check each connection against Circuit-1 once and fasten the wiring in place. The wiring is not changed again; every correction from this point is made in software.
4. Move the joystick supply. Move the 5 V and GND leads of the joystick from the Uno to the 5 V bus and the common ground.
5. Edit Code-2. Confirm that lines 55 to 60 give pins 5 to 10, and that `ledPin` (line 42) is the pin that the indicator LED is wired to: 12 in Circuit-1, or 3 in the advanced wiring (Uno Pin Assignments, Arduino pin numbers). Set line 92 to the first value of the enumeration:

```
motors.Bits(BitsL298N::bits_0000);
```

Select the Step2_JUL program in `platformio.ini`, line 31:

```
build_src_filter = +<*> -<Step1_Joystick/> +<Step2_JUL/>
```

Copy the joystick result. In the `updateJoystick()` function of Step2_JUL, lines 103 and 104, make the same changes to the two `analogRead()` lines that were made to Code-1 in Joystick Setup, steps 6 to 8, as recorded in the notebook. In `Common.h`, place comment marks on line 33 (`DEBUG_MAIN`) and remove them from line 43, so that it reads `#define DEBUG_L298N`; the L298N class then prints its four flags. Upload over USB.

```

33 void setup()
34 {
35   #ifdef DEBUG_SERIAL_ON
36     Serial.begin(9600);
37     while (!Serial) { }
38   #endif
39
40   // Button & Button-LED
41   int buttonPin = 2; // UNO D2 (CHIP-PD2)
42   int ledPin = 3;    // UNO D3 (CHIP-PD3)
43   // As indicated in article 1009 Guide
44   // int ledPin = 12; // UNO D12 (CHIP-PB4)
45   buttonDebug = Button(buttonPin, ledPin);
46
47   // Joystick Algorithm
48   joystick = Joystick<float>();
49
50   // Constructor setup the x-range and the y-range
51   mapInputFromDigital = LinearMap<float>(0, 1023, -1.0, 1.0);
52   mapOutFromJoystick = LinearMap<float>(-1.0, 1.0, -255, 255);
53
54   // Arduino PINS
55   int8_t ENA = 5;
56   int8_t IN1 = 6;
57   int8_t IN2 = 7;
58   int8_t IN3 = 8;
59   int8_t IN4 = 9;
60   int8_t ENB = 10;
61   // L298N PINS
62   int8_t LeftEN = ENA;
63   int8_t LeftA = IN1;
64   int8_t LeftB = IN2;
65   int8_t RightA = IN3;
66   int8_t RightB = IN4;
67   int8_t RightEN = ENB;
68
69   // L298N Constructor
70   motors = L298N(LeftEN, LeftA, LeftB,
71                 RightA, RightB, RightEN);
72   // Initiate L298N Pins
73   motors.PinsL298N();
74
75   // There are 16 combinations for EN, PWM, LeftIN & RightIN
76   // USE: analogWrite(EN, PWM) & digitalWrite(LeftIN1, LOW)...
77   // The Boolean Order (3, 2, 1, 0)
78   // (3) EN (AB): (Bit-3)
79   // (2) PWM Inputs (AB): (Bit-2)
80   // (1) LeftIN (1 & 2): (Bit-1)
81   // (0) RightIN (3 & 4): (Bit-0)
82   // EN A & B -----
83   // Bits-Value:  0    1    2    3    4    5    6    7
84   // Bits:        0000 0001 0010 0011 0100 0101 0110 0111
85   // EN B & A -----
86   // Bits-Value:  8    9   10   11   12   13   14   15
87   // Bits:        1000 1001 1010 1011 1100 1101 1110 1111
88   // -----
89   // NOTICE: Changing the BITS is much easier than switching
90   // the actual wires around on the L298N module....
91   // For my setup, bits 1010 was used...
92   motors.Bits(BitsL298N::bits_1010);
93 }

```

6. Apply power in order. With the Uno running on USB, apply the motor supply last, as in Safety and Supervision, item 4. PinsL298N() has already set the enable inputs LOW. With the variac supply, the supervising adult follows Using the Variac Supply in the Appendix.

7. Check the joystick again. With the grip at rest, turn the push button ON and confirm that the serial monitor prints L298N Input: a: 0 b: 0, the left and right motor commands, both zero. The joystick is now supplied by the 5 V bus, but the ADC of the Uno measures against its reference voltage, the 5 V supply of the Uno.[3] The reference voltage is the input voltage that the ADC reports as full scale, 1023. A command other than zero at rest means that the center reading has moved outside the dead zone; turn the push button OFF and repeat Joystick Setup, step 9. Turn the push button OFF.

8. Measure the motor bus. Perform Measuring the Motor Bus and the L298N Drop, steps 1 and 2, with the motors stopped. The loaded measurements, steps 3 to 8, are made after the Motor Movement Checklist, when the motors turn in the correct directions.

9. Find the value. Place each value of the enumeration in line 92 in turn, upload it, and check the motor movement with the Motor Movement Checklist, below. The first value that passes all eight joystick positions is the value for this wiring.

Table-3 lists the four flags encoded by Bits(), as defined in the current L298N class of Article 1003. The pattern is written as four binary digits in the order bit 3, bit 2, bit 1, bit 0, from bits_0000 to bits_1111.

Table-3

Bit	Name	Value 1	Value 0
3	EN	_EN_A paired with the left IN pair, _EN_B with the right	Enable pins crossed
2	PWM	Left IN pair follows the left command	Left and right commands swapped
1	LeftIN	Positive command sets IN_A LOW, IN_B HIGH	Positive command sets IN_A HIGH, IN_B LOW
0	RightIN	Same as bit 1, for the right IN pair	Same as bit 1, for the right IN pair

The four flags encoded by Bits(), from the current L298N class of Article 1003. Each BitsL298N value is written in the order bit 3, bit 2, bit 1, bit 0. Line 92 of Code-2, motors.Bits(BitsL298N::bits_1010), sets bit 3 = 1, enable pins straight; bit 2 = 0, left and right commands swapped; bit 1 = 1 and bit 0 = 0, opposite direction conventions for the left and right IN pairs. This is the value for the author's bench.

The author's bench uses bits_1010: straight enable pins, swapped commands, and opposite direction conventions for the two motors. Another bench wired differently needs a different pattern, and the checklist procedure of the Motor Movement Checklist finds it.

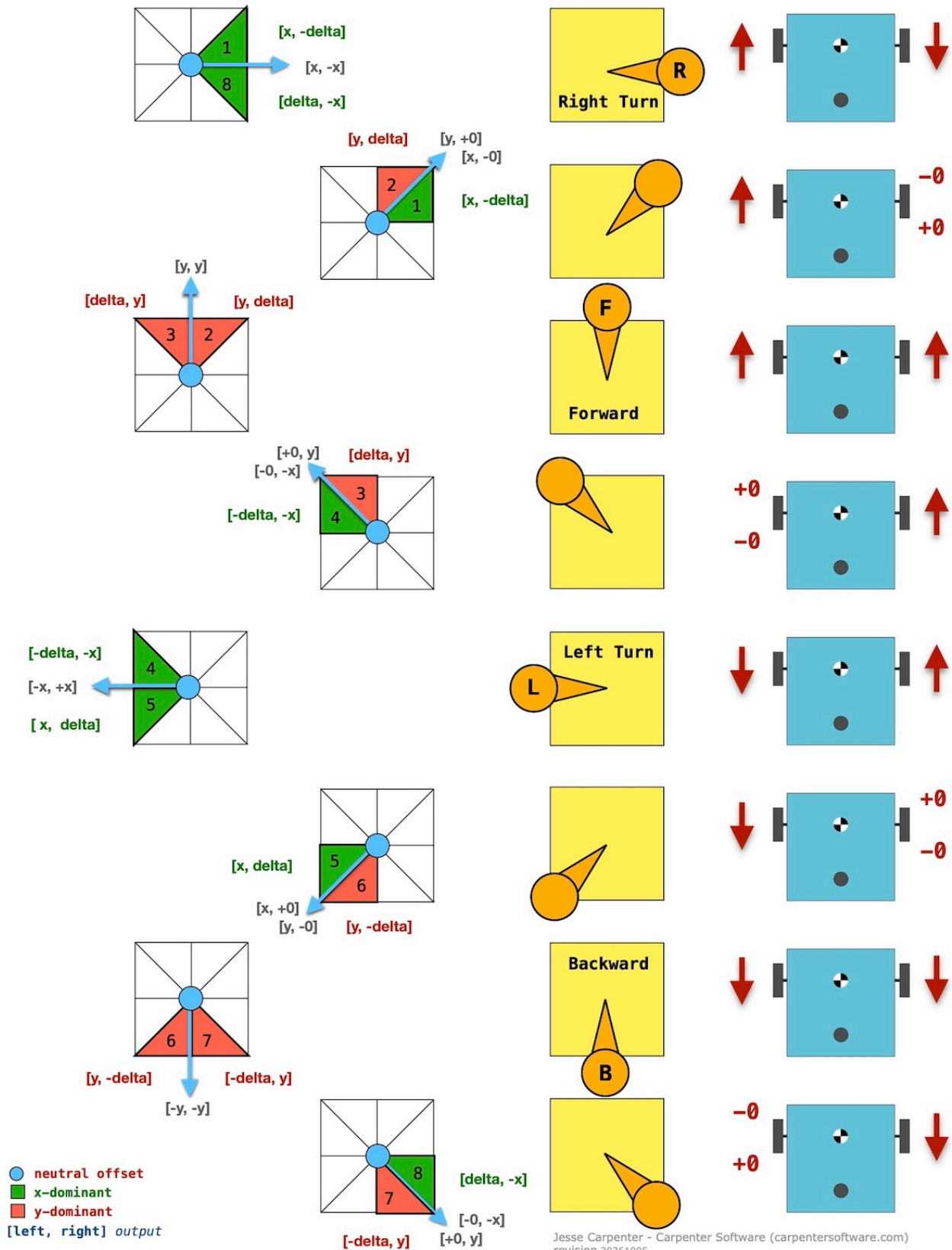
Motor Movement Checklist

The Motor Movement Checklist is the second half of L298N Setup. Each time a value of the BitsL298N enumeration is placed in line 92 of Code-2 and uploaded, the motor movement is checked with the checklist, and the student moves to the next value at the first wrong movement. The checklist uses three images: Image-5 gives the expected motor movement for each joystick position, Image-6 is the form the student fills in, and Image-7 is a completed example. The student studies all three before the checklist procedure begins.

Image-5, expected motor movement. Image-5 shows the eight joystick positions tested and the expected motor movement for each. Each test position lies on the boundary between two octant regions; Right Turn, for example, lies between regions 1 and 8. For every position, the left panel shows the two regions that meet at it, the center panel shows the grip position, and the right panel shows the expected direction of the left and right motors. An upward arrow is forward rotation, a downward arrow is reverse rotation, and +0 or -0 marks a motor that is stopped. On the checklist, the student records a stopped motor as a dash. The expected movement is written as [left, right]: 1 is forward at full command, -1 is reverse at full command, and 0 is stopped. Right Turn, [1, -1], therefore turns the left motor forward and the right motor in reverse.

Image-6, the blank checklist. Image-6 is the blank checklist, Revision 20261005b. There is one row for each of the sixteen Bits() values, bits_0000 to bits_1111, and one column pair for each of the eight test positions. Each test position lies on the boundary between two octants, and the column heading names it by those octants, from "Octants center of 8-1" to "Octants center of 7-8", counterclockwise from Right Turn. The expected output of each position is printed in the column heading as [left, right]: Right Turn, octants 8 and 1, [1, -1]; North-East, octants 1 and 2, [1, 0]; Forward, octants 2 and 3, [1, 1]; North-West, octants 3 and 4, [0, 1]; Left Turn, octants 4 and 5, [-1, 1]; South-West, octants 5 and 6, [-1, 0]; Backward, octants 6 and 7, [-1, -1]; and South-East, octants 7 and 8, [0, -1]. Each column pair has an INPUT column, where the student writes the four flags that the L298N class prints, and an OUTPUT column, where the student records the observed movement and circles the check mark or the X. Full size

Expected Motors Movement



Jesse Carpenter - Carpenter Software (carpentersoftware.com)
 revision 20261005
 ALL RIGHTS RESERVED

Motor Movement Checklist

M M C	INPUT R EN PWM Left IN Right IN	OUTPUT Octants center of 8-1 [1, -1] ↑ ↓	INPUT NE EN PWM Left IN Right IN	OUTPUT Octants center of 1-2 [1, 0] ↑ ↓	INPUT F EN PWM Left IN Right IN	OUTPUT Octants center of 2-3 [1, 1] ↑ ↑	INPUT NW EN PWM Left IN Right IN	OUTPUT Octants center of 3-4 [0, 1] ↑ ↓	INPUT L EN PWM Left IN Right IN	OUTPUT Octants center of 4-5 [-1, 1] ↑ ↓	INPUT SW EN PWM Left IN Right IN	OUTPUT Octants center of 5-6 [-1, 0] ↑ ↓	INPUT B EN PWM Left IN Right IN	OUTPUT Octants center of 6-7 [-1, -1] ↑ ↓	INPUT SE EN PWM Left IN Right IN	OUTPUT Octants center of 7-8 [0, -1] ↑ ↓
Bits	E	P	L	R	E	P	L	R	E	P	L	R	E	P	L	R
0 0000					✓ x				✓ x							✓ x
1 0001					✓ x				✓ x							✓ x
2 0010					✓ x				✓ x							✓ x
3 0011					✓ x				✓ x							✓ x
4 0100					✓ x				✓ x							✓ x
5 0101					✓ x				✓ x							✓ x
6 0110					✓ x				✓ x							✓ x
7 0111					✓ x				✓ x							✓ x
8 1000					✓ x				✓ x							✓ x
9 1001					✓ x				✓ x							✓ x
10 1010					✓ x				✓ x							✓ x
11 1011					✓ x				✓ x							✓ x
12 1100					✓ x				✓ x							✓ x
13 1101					✓ x				✓ x							✓ x
14 1110					✓ x				✓ x							✓ x
15 1111					✓ x				✓ x							✓ x

Instructions (Two observers: (1) programmer & (2) MMC – Lab Notebook)

INPUT: Write the four flags that the L298N class prints on the serial monitor (E, P, L, R). **OUTPUT:** Use the Expected Motor Movement Image (Image-5) to compare the output movement arrows.

(1) Do the vertical and horizontal movements first: R, F, L, & B. If the motor **OUTPUTS** match the arrows above, circle the check mark ✓. Continue to the next movement test.

(R) **Right Turn** between Octants 8-1 (F) **Forward** between Octants 2-3 (L) **Left Turn** between Octants 4-5 (B) **Backward** between Octants 6-7

(2) If a test fails, circle the X. STOP, then go to the next Bits value (next row) and return to step (1).

(3) If R, F, L, & B all pass, do the diagonals next: NE, NW, SW, & SE. If the motor **OUTPUTS** match the arrows above, circle the check mark ✓. Continue to the next movement test.

(NE) **North-East** between Octants 1-2 (NW) **North-West** between Octants 3-4 (SW) **South-West** between Octants 5-6 (SE) **South-East** between Octants 7-8

(4) If a test fails, circle the X. STOP, then go to the next Bits value (next row) and return to step (1).

(5) If all eight test positions of a row are circled with check marks ✓, stop: the correct motor behavior has been found.

copies of Image-5 and Image-6, in color and in black and white, with a completed example, are in 1009 Supplemental, Motor Movement Checklist (MMC), available for printing at [Link: Google Drive URL of 1009 Supplemental, Motor Movement Checklist (MMC)]. The link is published only in this guide, and the Supplemental carries the copyright notice of this guide.

Image-7, the author's completed checklist. Image-7 is the author's completed checklist for the Joystick-Uno-L298N bench, dated 20240811 by hand. It was recorded on an earlier layout of the form (Revision 20240811): the expected outputs in square brackets are the same as in Image-6, but the columns are numbered 1 to 8, and the column for $[1, -1]$ is marked (L) and the column for $[-1, 1]$ is marked (R), where Image-5 and Image-6 name these positions Right Turn and Left Turn. Most rows stop at the first column, and every row stops at its first circled X. Only the row bits_1010 has a check mark in all eight columns, so bits_1010 is the value for the author's wiring, the value in line 92 of Code-2. The note "23 out of 128" records that 23 of the 128 possible tests, sixteen rows of eight positions, were needed to find it.

Checklist procedure.

1. Confirm the start. L298N Setup, steps 1 to 8, is complete. Line 92 of Code-2 holds bits_0000, DEBUG_L298N is defined, the program is uploaded, the serial monitor is open, and the motor supply is on. The student has a printed blank copy of Image-6, from the Supplemental.
2. Do not record data while the joystick grip is at rest in the center. For each test position, push the grip to the edge of its travel in the direction shown in Image-5, and hold it there while both motors are observed. Between test positions, move the grip along the edge of its travel.
3. Test the four principal directions first: Right Turn (octants 8 and 1), Forward (octants 2 and 3), Left Turn (octants 4 and 5), and Backward (octants 6 and 7). Then test the diagonals: North-East (octants 1 and 2), North-West (octants 3 and 4), South-West (octants 5 and 6), and South-East (octants 7 and 8).
4. Hold the grip in the position shown in Image-5 until a new set of flags prints, up to 3 s, and observe both motors. In the INPUT column, write the four flags that the L298N class prints (E, P, L, R), as in Image-7. In the OUTPUT column, record the observed movement as arrows and dashes, beside the check mark and the X.
5. Compare the observed movement with the expected output printed in the column heading. If they match, circle the check mark. If they do not match, circle the X, and

record the difference between the observed movement (arrows and dashes) and the expected output (square brackets).

Image-7

20240811 te

MMC	Data		Data		Data		Data		Data		Data		Data		Data		Data			
	In	Out	In	Out	In	Out	In	Out	In	Out	In	Out	In	Out	In	Out	In	Out		
Bits	E	P	L	R	E	P	L	R	E	P	L	R	E	P	L	R	E	P	L	R
0 0000	B	B	1	2	1	2	1	2	1	2	1	2	1	2	1	2	1	2	1	2
1 0001	B	B	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	
2 0010	B	B	2	2	1	2	1	2	1	2	1	2	1	2	1	2	1	2	1	
3 0011	B	B	2	1	1	2	1	2	1	2	1	2	1	2	1	2	1	2	1	
4 0100	B	A	2	1	1	2	1	2	1	2	1	2	1	2	1	2	1	2	1	
5 0101	B	A	2	2	1	2	1	2	1	2	1	2	1	2	1	2	1	2	1	
6 0110	B	A	1	1	1	2	1	2	1	2	1	2	1	2	1	2	1	2	1	
7 0111	B	A	1	2	1	2	1	2	1	2	1	2	1	2	1	2	1	2	1	
8 1000	A	B	1	2	1	2	1	2	1	2	1	2	1	2	1	2	1	2	1	
9 1001	A	B	1	1	1	2	1	2	1	2	1	2	1	2	1	2	1	2	1	
10 1010	A	B	2	2	1	2	1	2	1	2	1	2	1	2	1	2	1	2	1	
11 1011	A	B	2	1	1	2	1	2	1	2	1	2	1	2	1	2	1	2	1	
12 1100																				
13 1101																				
14 1110																				
15 1111																				

Do not record data when the joystick is in a neutral position.
While recording data, position joystick in direction shown for both output and also used for input.
Do these tests first L, F, R, & B:
(L) Left Turn
(F) Forward
(R) Right Turn
(B) Backward
If the L, F, R, & B passes, then do the diagonals next.

Enter arrows & dashes for the movement - examples
Octant-1 horizontal [1, -1] = $\nwarrow$
Octant-2 diagonal [1, 0] = $\uparrow$
Octant-7 vertical [-1, -1] = $\swarrow$
Octant-8 diagonal [0, -1] = $\downarrow$

MMC - Motor Movement Checklist
If the movement ($\uparrow$) matches the expected test, circle the checkmark ($\checkmark$). If the expected test [1, 1] does not match, then circle the (X). Note the difference between movement and expected test. One has arrows and dashes. The other has square brackets. Once the test is circled with the (X), stop and go to the next line with the next bits value.

23 out of 128.

The author's completed checklist for the Joystick-Uno-L298N bench, on an earlier layout of the form (Revision 20240811). Row bits_1010 passes all eight positions; 23 of the 128 possible tests were needed to find it.

6. When an X is circled, stop testing that row. Change line 92 to the next value of the enumeration (bits_0000, then bits_0001, and so on to bits_1111), upload, and continue on the next row.

7. The correct value is the first row in which all eight test positions are circled with a check mark. Record it in the notebook and leave it in line 92.

8. Confirm. With the correct value uploaded, test all eight positions once more without changing anything, then complete Measuring the Motor Bus and the L298N Drop, steps 3 to 8. The supervisor signs the notebook entry as the corroborating witness. DEBUG_L298N may then be removed to stop the serial output.

9. If no row passes, the fault is not in the Bits() value. Turn off the motor supply, check the wiring against Circuit-1 and Table-2, repeat Joystick Setup, and consult Table-4.

Reading the INPUT column. With DEBUG_L298N defined, the UpdateL298N() function prints one line for each flag. E and P depend only on the Bits() pattern, so they are the same in every column of a row. L and R depend on the pattern and on the sign of the motor value, so they can change from one octant to the next. A motor value of 0 is treated as positive. After the four flags, the class prints the pattern and its value, for example bit-order(3210): 0000 val: 0, which confirms the value uploaded in line 92.

E, enable pins (bit 3). A, printed "(A) Bit-EN", when bit 3 is 1: the PWM value A drives the LeftEN pin (D5 in Code-2) and the PWM value B drives the RightEN pin (D10). B, printed "(B) Bit-EN", when bit 3 is 0: the two enable pins are exchanged.

P, PWM inputs (bit 2). A, printed "(A) Bit-PWM", when bit 2 is 1: the PWM value A is the left output of the joystick and the PWM value B is the right output. B, printed "(B) Bit-PWM", when bit 2 is 0: the two outputs are exchanged.

L, left input pins (bit 1). 1, printed "(1) LeftIn12": IN1 (D6) LOW and IN2 (D7) HIGH. 2, printed "(2) LeftIn21": IN1 HIGH and IN2 LOW. The digit is 1 when bit 1 is 1 and the PWM value A is positive, or when bit 1 is 0 and the PWM value A is negative; otherwise it is 2.

R, right input pins (bit 0). 1, printed "(1) RightIn34": IN3 (D8) LOW and IN4 (D9) HIGH. 2, printed "(2) RightIn43": IN3 HIGH and IN4 LOW. The digit follows bit 0 and the sign of the PWM value B in the same way.

An entry such as BB12 in Image-7 therefore reads E = B, P = B, L = 1, R = 2. The names Left and Right in the L298N class distinguish the two channels of the module only; which physical motor turns is set by the wiring and by E.

Troubleshooting

Table-4 lists the most common symptoms, the likely cause of each, and the section of this guide or of the series to revisit.

Table-4

Symptom	Likely cause	Revisit
No serial output	Baud rate not 9600, or debug flag off	Joystick Setup
Wrong axis changes	A0 and A1 exchanged	Joystick Setup, step 6
Indicator LED does not light	ledPin in Code-2 does not match the pin the LED is wired to (D12 or D3)	Uno Pin Assignments, Arduino pin numbers
Axis values reversed	Joystick mounted rotated	Joystick Setup, steps 7 and 8
Motor does not move	Enable wire or common ground missing	Safety item 3, Article 1003 Board Connections
Motor runs at full speed at any position	ENA or ENB jumper still installed	Article 1003 Board Connections
Robot turns the wrong way	Bits() pattern does not match the wiring	Motor Movement Checklist
Motor creeps or jitters at center	Dead zone too small	Article 1001, Article 1004
L298N heat sink very hot	Stall, high current, or high PWM frequency	Article 1003, PWM and Braking
Button state toggles by itself or not at all	SW circuit of the module does not match the Button setting (activeLow)	Joystick Setup, step 4
7805 very hot	Motor bus far above 12 V without a heat sink	Appendix, Circuit-2, the Schematic
Motor voltage well below the motor rating at full duty	Drop of the L298N not allowed for in the bus voltage	Measuring the Motor Bus and the L298N Drop

Common symptoms, the likely cause of each, and the section to revisit. One cause is corrected at a time, in the program where possible, and the test is repeated before the next change.

Appendix, Variable Bench Power Supply with Variac

Purpose of the Variac Supply

The Electronics Lab describes two motor supplies, a fixed supply and a variable supply. The fixed supply gives one bus voltage, set by its transformer. The variable supply gives any bus voltage from 0 V to its limit, so one supply serves motors of

any rating, and the bus can be raised in small steps while the drop of the L298N is measured. This appendix describes the variable supply, which Article 1000 also introduces. The variac, the variable autotransformer of Safety and Supervision, item 2, is a single winding on a toroidal core with a sliding brush, so that its output is a selectable fraction of its input. Image-8a shows the supply in three panels, from top to bottom: the components in order, a model of the supply on a bench board, and the schematic, Circuit-2.

Why study this supply. A linear supply is built from parts that can be seen and measured one at a time: the transformer sets the voltage by its turns ratio, the bridge rectifier converts AC into pulsating DC, the filter capacitor smooths it, and the bleed resistor discharges it. Each stage has a voltage that the multimeter reads, so the student observes what each part does. The variac adds one variable: the bus can be set to any voltage and changed in small steps, so the student observes how the motor speed, the drop of the L298N, the fall of the bus under load, and the input of the 7805 change with the bus voltage. A commercial bench supply does the same work inside a closed case, where none of these stages can be observed.

Why it is in the Appendix. The supply connects to the AC line, so it is built, adjusted, and switched by the supervising adult, not by the student, and the questions it answers are advanced. The guide refers to it wherever such a question arises, and Advanced Questions with the Variable Supply, at the end of this appendix, collects them.

Circuit-2, the Schematic

Circuit-2, labels, from left to right. AC line, 120 VAC, drawn as a sine wave source. Variac: autotransformer, a single winding with a sliding brush, drawn as an arrow, 0 to 120 VAC, not isolated from the AC line. Center tap step down transformer: turns ratio 4:1, 0 to 30 VAC across the full secondary; the primary winding (left of the core) has N_p turns and the secondary winding (right of the core) has N_s turns, and the two lines between the windings are the iron core. The secondary has three terminals: two outer terminals (yellow) and the center tap (green), a connection at the midpoint of the secondary winding, which is not connected in this supply. Bridge rectifier: four diodes; the AC inputs are the left and right nodes, the + output is the upper node, and the – output is the lower node. The small arc where the lower yellow lead passes the – node marks a crossing without a connection. Filter capacitor: electrolytic, polarized, with + on the upper plate. Bleed resistor: discharges the capacitor. The red terminal at the right is the + DC output (upper rail), and the black terminal is the – DC output (lower rail).

Circuit-2, description. The AC line (120 VAC) feeds the variac, which delivers an adjustable 0 to 120 VAC from its brush. The variac output drives the primary of a step down transformer with a turns ratio of 4:1, the ratio of primary turns (N_p) to secondary turns (N_s), so the full secondary delivers 0 to 30 VAC. The two outer

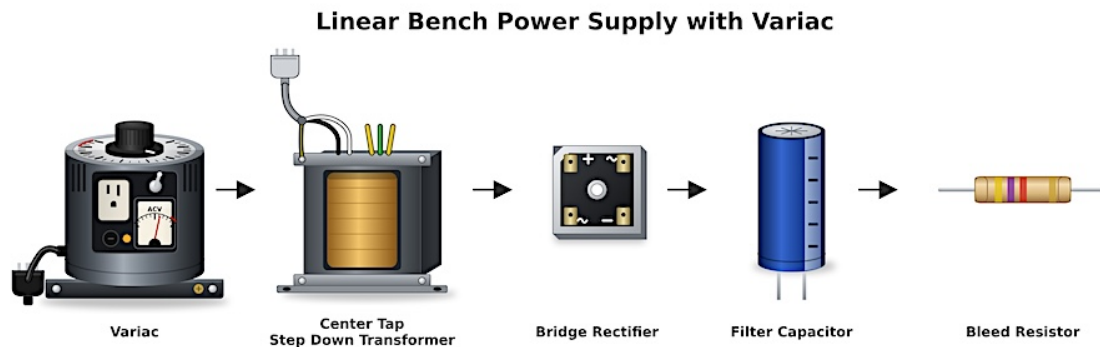
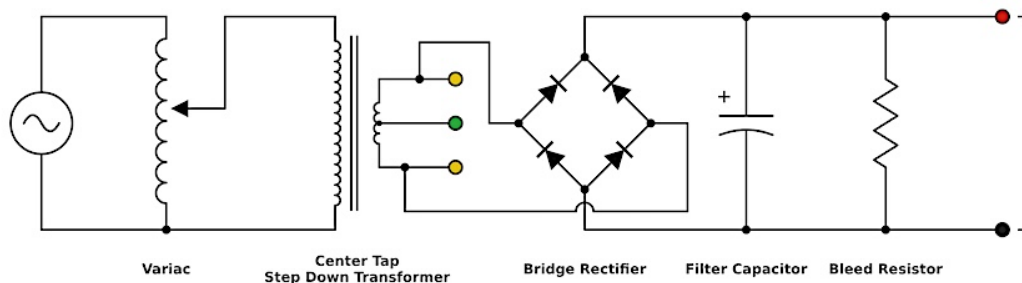
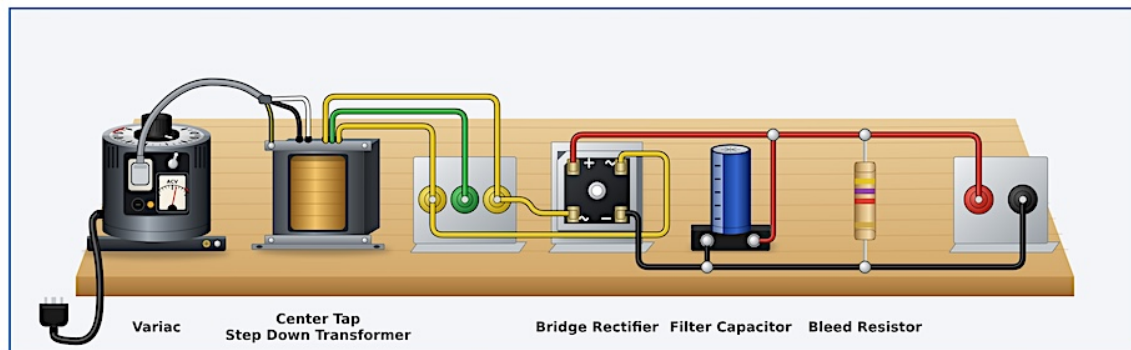


Image-8a



Circuit-2

Components. The parts of the supply in the order of the power flow: the variac, the center tap step down transformer, the full wave bridge rectifier, the filter capacitor, and the bleed resistor.

Diagram. The supply assembled on a wooden bench board, described under Image-8a, the Bench Model.

Circuit. Circuit-2, the schematic of the supply, described under Circuit-2, the Schematic.

See Image-8b, **Safety Precautions.**

terminals of the secondary feed the AC inputs of the full wave bridge rectifier, which converts the secondary voltage to pulsating DC; the center tap is not connected. The filter capacitor charges to nearly the peak of each pulse and smooths the output. The bleed resistor across the capacitor discharges it when the supply is turned off. The transformer core is the boundary between the AC line side, to its left, and the isolated side, to its right. The voltages and the turns ratio are not printed in Circuit-2; they are the design values used in the calculations below. Circuit-2 ends at the DC motor bus. The 7805 regulator and the 5 V bus are part of Circuit-1 and connect to the output binding posts; they are not built into this supply. The absolute maximum input of the 7805 is 35 V, the same limit as the capacitors of the L298N module, so the variac limit in the calculations below protects both. The 7805 dissipates $(V_{\text{bus}} - 5) \times I$ as heat; at a 24 V bus and 100 mA it dissipates 1.9 W and requires a heat sink.

Circuit-2, calculations. The peak of a sine wave is 1.414 times its RMS value. At 30 VAC the peak is $30 \times 1.414 = 42.4$ V, and the two conducting diodes of the bridge drop about 1.4 V, so the unloaded DC output is about 41 V. The capacitors on the L298N module are rated at 35 V, so for the Joystick-Uno-L298N project the secondary must not exceed $(35 + 1.4) / 1.414 = 25.7$ VAC, which is a variac setting of about 103 VAC. The ripple voltage is approximately $I / (2 \times f \times C)$; at 1 A, 60 Hz, and 4700 μF it is $1 / (2 \times 60 \times 0.0047) = 1.8$ V. The time constant of the bleed resistor and capacitor is $R \times C = 4700 \, \Omega \times 0.0047 \, \text{F} = 22 \, \text{s}$, and the capacitor falls below 1 percent of its voltage in about five time constants, or 110 s. At 41 V the bleed resistor dissipates $41 \times 41 / 4700 = 0.36$ W, so a 1 W resistor is used.

Circuit-2, safety precautions. Image-8b lists twelve precautions for building and operating the variac supply. Precautions 9 and 11 follow the engineering data of the variac manufacturer.[4] Image-8a, description, refers to precautions 2 and 3 by number.

Image-8a, the Bench Model

Image-8a, description. The middle panel of Image-8a is a three dimensional model of the variac supply as assembled on a wooden bench board. From left to right: the line cord of the variac plugs into the AC line; a gray cord plugged into the outlet on the variac carries the variac output, as a black lead and a white lead, to the transformer primary; two yellow leads from the outer terminals of the secondary and a green lead from the center tap run to a terminal panel of three binding posts; the two yellow binding posts feed the $\sim$ terminals of the bridge rectifier, which is mounted on an aluminum plate, and the green binding post is not

Safety Precautions

Image-8b

1. Adult supervision is required. Students work only on the isolated DC side, to the right of the transformer.
2. The variac is an autotransformer: its output shares a conductor with the AC line. Every terminal on the AC line side can be at line potential. Enclose the variac, transformer primary, fuse, and switch in a grounded metal or insulated case.
3. Use a three wire, grounded line cord. Fuse the hot conductor and switch it before the variac. Bond all metal frames to earth ground. The fuse and switch are not drawn in Circuit-2.
4. Unplug the line cord, not only the switch, before any wiring change. Turning the switch off does not remove line voltage from the cord.
5. Set the variac to zero before power is applied. Raise it slowly while the DC output is measured with a multimeter.
6. The filter capacitor stores charge after power is removed. The bleed resistor discharges it. Measure 0 V across the capacitor with a multimeter before touching the circuit.
7. Observe capacitor polarity. A reversed electrolytic capacitor can vent or rupture. Rate the capacitor above the peak DC output: $30 \text{ VAC} \times 1.414 \approx 42 \text{ V peak}$, so use a 63 V capacitor. Do not exceed 35 V on an L298N module, whose capacitors are rated at 35 V.
8. The transformer, rectifier, and bleed resistor become hot under load. Keep flammable material away from the supply.
9. The output of the variac is not isolated at any setting. A low setting of the knob does not make the output terminals safe to touch, because each terminal can still be at line potential with respect to earth ground.[4]
10. Never connect a grounded instrument, such as the ground clip of an oscilloscope probe, to the variac output or to the transformer primary. The ground of the instrument would short the AC line to earth ground. All measurements are made on the isolated DC side, at the output binding posts.
11. Connect the common terminal of the variac to the neutral conductor of the line cord, and the input terminal to the fused and switched hot conductor; the brush terminal then carries the adjustable output.[4] A second fuse in the brush output lead protects the variac from overload.[4]
12. The step down transformer of Circuit-2 provides the galvanic isolation of the DC side. A variac with an isolated winding, or an isolation transformer ahead of the variac, also isolates its output, but neither replaces the enclosure, the fuse, and the grounding of precautions 2 and 3.

connected, as in Circuit-2; a red lead runs from the + terminal of the bridge and a black lead from the – terminal to the two terminals at the base of the filter capacitor; the bleed resistor is connected across the red and black leads; and the leads end at the red (+) and black (–) binding posts on the aluminum output panel. The color convention is black and white for the AC line, yellow for the outer terminals of the transformer secondary, green for its center tap, red for positive DC, and black for negative DC. Image-8a does not show the fuse, the switch, the bonding of the metal frames to earth ground, or the enclosure required by Circuit-2, safety precautions 2 and 3.

Image-8a, safety notes.

1. Wires on the AC line side use insulation rated for line voltage, and no bare conductor is exposed; the exposed terminals in Image-8a are shown for clarity and are covered in a finished supply.
2. Wires are routed flat along the board and away from the heat sink and the transformer.
3. The output binding posts are the only connection a student makes to this supply.

Using the Variac Supply

The supervising adult performs every step. The student reads the multimeter and records the values.

1. Inspect. The line cord is unplugged. The variac, transformer primary, fuse, and switch are enclosed, the line cord is three wire and grounded, and the filter capacitor is connected with the correct polarity.
2. Set to zero. Turn the variac knob fully counterclockwise, to 0 VAC.
3. Disconnect the load. The output binding posts are not connected to the bench. Connect the multimeter, set to DC volts, across the output binding posts.
4. Apply power. Plug in the line cord and turn on the switch. The meter reads 0 V.
5. Raise the bus. Turn the variac slowly while watching the meter, and stop at the motor rating, for example 12 V for 12 V motors (a variac setting of about $4 \times (12 + 1.4) / 1.414 = 38$ VAC). Never exceed 35 V at the output, the limit of the L298N module and the 7805; the variac limit is about 103 VAC.

6. Turn off and connect. Turn the variac to zero, turn off the switch, and unplug the line cord. Wait two minutes and measure 0 V at the output. Connect the red (+) binding post to the motor bus and the black (–) binding post to the common ground of the bench.

7. Set the bus under load. Plug in, turn on the switch, and raise the variac to the motor rating again. Perform Measuring the Motor Bus and the L298N Drop (The Electronics Lab), and raise the bus by the measured drop as in its step 7.

8. Shut down. Turn the variac to zero, turn off the switch, and unplug the line cord. Wait two minutes, about five time constants of the bleed resistor and the filter capacitor, and measure 0 V across the capacitor before touching the circuit.

Advanced Questions with the Variable Supply

Each question below is answered with the variable supply and the multimeter, by the student under the supervision of the adult who operates the supply. Article 1004 may present some of them as experiments.

1. Fall of the bus under load. How far does the bus fall from no load to the motors running at full duty? (Measuring the Motor Bus and the L298N Drop, steps 2 and 4.)
2. Drop of the L298N. How does the drop change between light load and heavy load? Compare each value with the datasheet range.[2]
3. Ripple. How does the ripple change with the motor current? Compare the measured fall of the bus with the calculated ripple, $I / (2 \times f \times C)$, in Circuit-2, calculations.
4. Input limit of the 7805. With the motors stopped, lower the bus slowly and record the bus voltage at which the 5 V bus begins to fall below 5 V.
5. Motor speed and voltage. Raise the bus in steps of 1 V, up to the motor rating plus the drop. At each step, count the revolutions of one wheel in 10 s and plot revolutions per minute against the motor voltage.

A Note on Software and Firmware

The words software and firmware are often used interchangeably for the programs of a microcontroller, and other references the student reads may use either one. Software is the general term for the instructions that a computer or a microcontroller executes. Firmware is software stored in the nonvolatile memory of a device, memory that keeps its contents when power is removed, such as the flash memory of the ATmega328P, and it runs each time the device is powered, without an operating system. By these definitions the programs

that the student uploads to the Uno are both software and firmware. For a consistent usage, this guide and the other articles of the series use the word software.

Bibliography

[1] Carpenter, Jesse. "Joystick-Uno-L298N." MageMCU, GitHub repository. Accessed September 23, 2026. <https://github.com/MageMCU/Joystick-Uno-L298N>.

[2] STMicroelectronics. "L298 Dual Full-Bridge Driver." Datasheet DS0218, Rev. 5. October 2023. Accessed September 21, 2026. <https://www.st.com/resource/en/datasheet/l298.pdf>.

[3] Microchip Technology Inc. "ATmega48A/PA/88A/PA/168A/PA/328/P Data Sheet Complete." Document DS40002061B. 2020. Accessed September 22, 2026. <https://www.microchip.com/content/dam/mchp/documents/MCU08/ProductDocuments/DataSheets/ATmega48A-PA-88A-PA-168A-PA-328P-DS-DS40002061B.pdf>.

[4] STACO Energy Products Co. "Variable Transformers: Product Design and Engineering Data." Accessed October 3, 2026. <https://variac.com/staco/PDFCutSheets/VT%20designengine.pdf>.

Disclaimer and License

- Version 2.0, Last Updated: September 24, 2026 by Jesse Carpenter

Carpenter Software and the MageMCU account at GitHub

Software: Use It Freely, Just Give Credit

The software in the MageMCU repositories on GitHub is free to use, copy, modify, share, and build upon, including for commercial purposes, under the MIT License. The only thing asked in return is credit. Articles and other content are not covered by this permission; see item 3.

How to Give Credit

When you use or share the Software, please include a line such as:

Based on work by Jesse Carpenter, Carpenter Software (carpentersoftware.com), MageMCU on GitHub (github.com/MageMCU).

If you changed the Software, please say so (for example, "adapted from" or "modified from").

What Is Covered

1. Any information created or presented by Jesse Carpenter (the "AUTHOR") on GitHub, through Carpenter Software, or elsewhere, including instructional information, electronic schematics, and software, in any digital or non-digital format, is collectively referred to as the "**Materials**."
2. **Software** (source code in the MageMCU repositories on GitHub) is licensed under the MIT License below. Articles, tutorials, and schematics stored in the same repositories are not part of the Software and are not licensed under the MIT License.
3. **Articles and other content**, including tutorials, schematics, diagrams, images, posts, genealogy records, family stories, books, and all other content created by the AUTHOR, wherever published (including content stored in a MageMCU repository), are protected by copyright. All rights reserved. No part may be reproduced, republished, or distributed without the written permission of the AUTHOR, except as permitted in item 7 or by copyright law (for example, brief quotations with attribution for review, commentary, or teaching). Readers may apply the information in the articles, for example by building the circuits or following the instructions, at their own risk as described in items 4 and 5. Copyright restricts copying the articles, not applying what they teach.

No Warranty, No Liability

4. The Materials are provided "as is," without warranty of any kind, expressed or implied, including any warranty of accuracy, reliability, performance, or fitness for a particular purpose.
5. Anyone who uses the Materials or applies the information they contain (the "USER") assumes the entire risk of that use. To the maximum extent permitted by applicable law, the AUTHOR and Carpenter Software are not liable for any damages of any kind arising from the use of, or inability to use, the Materials.
6. The MageMCU GitHub account is the AUTHOR's working space. Its repositories are works in progress; they may be incomplete, may contain errors, and may change or be removed without notice. The AUTHOR has no obligation to provide support, updates, or corrections.

Teaching

7. Teachers, parents, and mentors may use the Software freely and may reference, link to, and display the articles during instruction. They may also print or share copies of articles with their own students for noncommercial educational use only, provided the copyright notice remains intact. Any other reproduction or distribution requires the written permission of the AUTHOR. When teaching, please share the safety notices below with students and, where minors participate, with their parents or guardians.

Parents and Teachers

The Materials in this GitHub account are not intended for unsupervised use by minors. When working with electricity of any kind, minors must be directed and instructed by a responsible adult mentor. Anyone using GitHub must meet GitHub's minimum age requirement (13 years of age in the United States; older in some countries).

Students and Young Builders

All children and teenagers: talk with your parent, guardian, mentor, or teacher before experimenting with any of these electronic circuits. Never work with electricity alone. **Electricity Kills.** See the notice below.

NOTICE: Keep flammable materials, liquids, and loose metal objects away from power supplies and powered circuit boards. Use a power supply with an enclosed case to protect minors. When instructing young students, keep power sources out of reach and follow a hands off approach while circuits are powered. Horseplay is not permitted and is grounds for removal from the activity.

Additional hazards common to microcontroller and robotics projects:

- **Batteries:** Lithium polymer (LiPo) and lithium ion batteries can overheat, swell, or catch fire if shorted, punctured, or charged improperly. Use the correct charger and never leave a charging battery unattended.
- **Short circuits:** A shorted battery or supply can make wires and components hot enough to burn skin.
- **Soldering:** Soldering irons reach temperatures that cause burns. Work in a ventilated area and use a proper iron stand.
- **Moving parts:** Motors, wheels, and gears can pinch fingers. Disconnect power before handling a robot's drive system.
- **Eye protection:** Wear safety glasses when soldering or trimming component leads.

MIT License (Software)

Copyright © 2021-2026 Jesse Carpenter (Carpenter Software), MageMCU at GitHub

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Copyright Notice (Articles and Other Content)

Copyright © 2021-2026 Jesse Carpenter, Carpenter Software. All rights reserved.

Teachers, parents, and mentors may print copies for their own students for noncommercial educational use, with this notice intact (Disclaimer and License, item 7, MageMCU on GitHub). Any other reproduction or distribution requires the written permission of the author.

Correspondence

Jesse Carpenter (Robotics)
1025 Chelwood Park Blvd NE, Apt 102
Albuquerque, NM 87112 USA
Phone (505) 503-1129
Email: hello@carpentersoftware.com

Website: Carpenter Software
carpentersoftware.com



1009 Parent, Teacher, and Student Guide to Joystick-Uno-L298N

STEM Starter Kit Series, Companion to Articles 1000 to 1004 (Article 1009)

Created: 1009 L298N Supplemental (20230115)

Revised: 1009 L298N Supplemental (20240820)

Revised: 1009 Parent, Teacher, and Student Guide to Joystick-Uno-L298N (20261007b)

by Jesse Carpenter (Carpenter Software)

Article ongoing revisions.

ALL RIGHTS RESERVED